

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет информатики, математики и компьютерных наук

Лекомцев Павел Олегович

**Разработка интеллектуальной системы управления и мониторинга
парковочного пространства на основе компьютерного зрения**

Выпускная квалификационная работа - БАКАЛАВРСКАЯ РАБОТА
по направлению подготовки 01.03.02 Прикладная математика и информатика
образовательная программа
«Прикладная математика и информатика»

Рецензент

Руководитель

к. ф. – м. н., старший преподаватель,
Кочеганов Виктор Михайлович

Нижний Новгород, 2026

Оглавление

Введение.....	5
Глава 1. Анализ задачи управления парковочным пространством и обзор существующих подходов.....	10
1.1. Проблема управления парковочным пространством в современном городе	10
1.2. Классификация методов контроля парковки	12
1.3. Обзор существующих систем умного паркинга	15
1.3.1. Система Nedap SENSIT	15
1.3.2. Система ParkAssist M4	16
1.3.3. Российские решения в сегменте платных парковок	17
1.3.4. Сравнительный анализ и общие тенденции.....	18
1.4. Методы компьютерного зрения для задач мониторинга транспорта	20
1.4.1. Архитектуры однопроходной детекции объектов	20
1.4.2. Методы многообъектного трекинга	22
1.4.3. Распознавание государственных регистрационных знаков	24
1.4.4. Геометрические методы анализа положения автомобиля.....	25
1.5. Постановка задачи исследования	26
Глава 2. Подготовка данных и разработка моделей машинного обучения	29
2.1. Проектирование виртуального полигона в Unreal Engine 5	29
2.1.1. Обоснование использования синтетических данных	29
2.1.2. Моделирование участка улицы Богородского и территории ТЦ «Нагорный»	30
2.1.3. Система автономных агентов	32
2.1.4. Суточный цикл и вариативность освещения сцены	34
2.1.5. Размещение и параметризация шести камер.....	35
2.1.6. Интеграция виртуальной сцены с серверной частью	36

2.2. Сбор и подготовка датасета	38
2.2.1. Пайплайн сбора кадров из сцены	38
2.2.2. Разметка изображений в CVAT	38
2.2.3. Разделение выборки и контроль утечек данных	39
2.2.4. Разведочный анализ данных	40
2.2.5. Документирование границ применимости	41
2.3. Обучение модели детекции автомобилей	42
2.3.1. Выбор базовой архитектуры	42
2.3.2. MLOps-инфраструктура на базе ClearML	42
2.3.3. Постановка и проведение трёх экспериментов	43
2.3.4. Сравнительный анализ метрик	43
2.3.5. Перенос чекпоинта в продакшн	44
2.4. Распознавание государственных регистрационных знаков	45
2.4.1. Двухстадийная архитектура PlateScanner	45
2.4.2. Выбор OCR-модели ParseQ	46
2.4.3. Конвертация и квантизация модели для edge-развёртывания ...	47
Глава 3. Проектирование и разработка системы мониторинга	
парковочного пространства	48
3.1. Формирование требований к системе	48
3.1.1. Функциональные требования	48
3.1.2. Нефункциональные требования	50
3.2. Общая архитектура и зонирование	52
3.2.1. Многоуровневая схема потоков данных	52
3.2.2. Распределение камер по функциональным зонам	54
3.3. Модуль детекции и определения занятости мест	55
3.4. Модуль обнаружения неправильной парковки	57
3.5. Модуль трекинга и детекции подозрительной стоянки	61
3.6. Модуль подсчёта транспортного потока	65
3.7. Модуль управления входным шлагбаумом	66

3.7.1. Двухкамерная схема наблюдения.....	67
3.7.2. Разметка зон приближения, считывания и безопасности.....	68
3.7.3. Конечный автомат барьерного контроллера.....	69
3.7.4. Консенсусный алгоритм распознавания номера.....	70
3.7.5. Окно ожидания зоны безопасности.....	71
3.7.6. Интеграция с Unreal Engine 5.....	72
3.7.7. Схема базы данных доступа.....	73
3.8. Серверная часть	75
3.8.1. Стек технологий и жизненный цикл приложения	75
3.8.2. REST API и WebSocket-интерфейс	75
3.8.3. Типизированная конфигурация	76
3.8.4. Структурированное логирование	76
3.9. Клиентская часть	76
3.9.1. Архитектура фронтенд-приложения	76
3.9.2. Агрегация статистики и дедупликация камер.....	77
3.9.3. Панель управления шлагбаумом и журнал доступа	77
3.10. Инфраструктура развёртывания и обеспечение качества	78
3.10.1. Контейнеризация и оркестрация.....	78
3.10.2. Непрерывная интеграция	78
3.10.3. Модульное и интеграционное тестирование.....	79
Заключение	80
Библиографический список.....	83
Приложение А. Дополнительные скриншоты виртуальной сцены и веб-интерфейса	87
Приложение Б. Расширенные таблицы результатов экспериментов ..	92
Приложение В. Ключевые фрагменты программного кода.....	93

ВВЕДЕНИЕ

Автомобилизация российских городов за последнее десятилетие перешла черту, за которой инфраструктура уже не успевает за парком. По данным Росстата, к началу 2026 года в Российской Федерации зарегистрировано около 52,8 млн легковых автомобилей, а обеспеченность - 338 машин на 1000 жителей. Нижегородская область относится к регионам-«миллионникам» по количеству транспорта; показатель обеспеченности в Нижнем Новгороде к 2024 году достиг 400 автомобилей на 1000 человек, что в полтора раза превышает планку, при которой проектировались советские жилые кварталы и центральные улицы города.

Следствие известно каждому жителю и водителю в районе улицы Богородского: крупные торговые центры окружены стихийной парковкой, часть машин стоит с заездом на тротуар или поперёк разметки, а проверка оплаты и допуска на закрытые площадки опирается на охрану с планшетом. В Нижнем Новгороде под платную парковку сейчас размечено около 7,4 тыс. мест по 119 адресам, но у частных объектов - ТЦ, офисных зданий, фитнес-клубов - парковочные зоны живут по собственным правилам и без аналитики. Для магистралей нагрузка уже запредельная: проспект Гагарина проходит через ТЦ «Нагорный» и принимает до 85 тыс. автомобилей в сутки. Ручной контроль в таких условиях не масштабируется, а датчики с заглублением в асфальт требуют капитальных работ и плохо переносят зимний цикл.

Сценарий с видеоаналитикой выглядит разумной альтернативой: камеры уже установлены ради охраны периметра, вычисления можно вынести на единый сервер, а алгоритмы детекции и трекинга в последние годы достигли качества, пригодного для реального времени. Остаётся инженерная задача - собрать из готовых моделей работоспособную систему, которая покрывает весь цикл от въезда через шлагбаум до подсчёта транспортного потока на прилегающей улице. Именно такая система реализована в

настоящей работе на примере участка улицы Богородского и торгового центра «Нагорный».

Цель работы - разработать интеллектуальную многокамерную систему управления и мониторинга парковочного пространства на основе компьютерного зрения, объединяющую в единый real-time пайплайн детекцию транспортных средств, оценку занятости парковочных мест, определение неправильной парковки, многообъектный трекинг для выявления подозрительно длительной стоянки, подсчёт транспортного потока с определением направления движения и автоматизированное управление шлагбаумом с распознаванием государственных регистрационных знаков, развёрнутую на стандартном серверном стенде и верифицированную на синтетическом цифровом двойнике городского участка.

Для достижения поставленной цели в работе решены следующие **задачи**:

1. Выполнен анализ существующих систем управления парковкой (сенсорных, видеоаналитических, комбинированных) и обоснован выбор нейросетевого стека на основе YOLOv8 для детекции автомобилей, SORT для многообъектного трекинга и связки YOLO11x + ParseQ для распознавания номерных знаков;

2. Спроектирован виртуальный полигон в Unreal Engine 5, воспроизводящий участок улицы Богородского и торговый центр «Нагорный», с автономными AI-агентами-автомобилями, пешеходом, суточным циклом освещения и шестью камерами наблюдения, образующими четыре функциональные зоны (бесплатная парковка, платная парковка, дорога, шлагбаум);

3. Собран и размечен в CVAT синтетический датасет из 822 кадров (1821 аннотация класса car), разбит на train/val/test в пропорции 70/20/10, подготовлены data card и EDA-отчёт;

4. Проведены три эксперимента обучения детектора автомобилей (baseline, aug_schedule, full_stack) на локальном сервере ClearML; лучшая конфигурация закреплена как production-модель «Car_Detector.pt» с метриками mAP50 = 0,9945 и mAP50-95 = 0,9738;

5. Реализованы шесть модулей компьютерного зрения, образующих ядро веб-приложения: (а) детекция автомобилей и оценка занятости парковочных мест по перекрытию bounding-box и полигона - для бесплатной и платной зон; (б) определение неправильной парковки на основе процента выхода bbox за границы места с адаптивными порогами под fisheye-искажения - для бесплатной и платной зон; (в) многообъектный трекинг SORT (фильтр Калмана + венгерский алгоритм сопоставления по IoU) - для платной зоны и дороги; (г) детектор подозрительной парковки по длительности нахождения трека в пределах места с порогом 30 секунд - для платной зоны; (д) подсчёт транспортного потока с определением направления (incoming / outgoing) по двум калибровочным полигонам - для дороги рядом с парковкой; (е) умный шлагбаум на базе конечного автомата из девяти состояний с консенсусным OCR (PLATE_MIN_AGREEING_READS = 3) и safety-зоной - для въезда на платную парковку;

6. Разработаны серверная часть на FastAPI с WebSocket-трансляцией кадров и статистики, SQLite-база barrier.db (таблицы allowed_plates, access_log, parking_sessions) и клиентский дашборд с зонированной навигацией, панелью управления шлагбаумом и CRUD-управлением списком разрешённых номеров;

7. Собрана инфраструктура развёртывания и обеспечения качества: Dockerfile и docker-compose для одно-командного запуска, GitHub Actions CI с ruff, mypy и pytest, pre-commit hooks, pytest-сьюта из 55 тестов в 6 файлах, покрывающих конфиг, SORT, стейт-машину шлагбаума, SQLite-слой, геометрию wrong parking и API;

8. Проведена квантизация модели распознавания номерных знаков YOLO11x в TFLite FP16, подтверждающая возможность переноса системы на edge-устройства без переобучения.

Объектом исследования является парковочное пространство торгово-офисного назначения и прилегающая к нему улично-дорожная сеть городского типа. **Предметом исследования** выступают методы и алгоритмы компьютерного зрения, применимые к задачам многокамерного мониторинга парковки в режиме реального времени.

Научная новизна работы заключается в трёх элементах. Во-первых, предложена зонированная архитектура многокамерной системы с адаптивными порогами *wrong parking*, компенсирующими fisheye-искажения без полноценной геометрической калибровки камер. Во-вторых, реализован консенсусный OCR с повторами на каждом кадре в пределах зоны чтения, снижающий влияние мерцания распознавания на решение о допуске. В-третьих, предложен двухкамерный паттерн управления шлагбаумом с *safety-zone grace period* в 3 секунды, устраняющий ложные закрытия барьера во время проезда автомобиля.

Практическая значимость состоит в том, что система поддерживает горизонтальное масштабирование на произвольное число камер через конфигурацию, а все артефакты - веса моделей, калибровочные файлы, база данных, CI-пайплайн - хранятся в одном репозитории. Верификация на цифровом двойнике позволяет измерять качество до развёртывания на реальный объект и сравнивать варианты моделей в контролируемых условиях.

В работе применены следующие методы: нейросетевая детекция объектов (архитектура YOLOv8), фильтрация Калмана и венгерский алгоритм сопоставления для многообъектного трекинга, пиксельно-геометрический анализ перекрытия bounding-box и полигона, моделирование поведения барьера конечным автоматом, контейнеризация Docker и непрерывная

интеграция GitHub Actions, синтетическая генерация обучающих данных в Unreal Engine 5.

ГЛАВА 1. АНАЛИЗ ЗАДАЧИ УПРАВЛЕНИЯ ПАРКОВОЧНЫМ ПРОСТРАНСТВОМ И ОБЗОР СУЩЕСТВУЮЩИХ ПОДХОДОВ

1.1. Проблема управления парковочным пространством в современном городе

Сформулированная во введении диспропорция между растущим легковым парком и дефицитом организованных мест для хранения автомобилей имеет несколько независимых источников, и каждый из них по отдельности уже диктует определённые требования к системе мониторинга.

Первый источник - инерция градостроительных норм. Нормативы СНиП 2.07.01-89, по которым проектировалась большая часть советских жилых кварталов и общественных центров, предполагали 150–200 машино-мест на 1000 жителей жилой застройки и 15–20 мест на 100 посетителей крупного торгового объекта. Обновлённый СП 42.13330.2016 поднял эти показатели до 420 машино-мест на 1000 жителей для многоквартирной застройки, что ближе к фактической автомобилизации, однако уже построенная городская ткань под новые нормы не перестраивается. В итоге жилые массивы и торговые центры советской и ранней постсоветской застройки вынужденно обслуживают парк, вдвое-втрое превышающий проектный.

Второй источник - неравномерность загрузки. Торговые центры в спальных районах, офисные комплексы в деловых кластерах, лечебные учреждения и вокзалы перекрывают пиковые нагрузки совершенно разными способами. Пиковый коэффициент спроса на парковочные места у ТЦ общей площадью около 40 000 м² достигает 0,7–0,8 от общей ёмкости в будний вечер и 0,95 в субботу после полудня, тогда как средний уровень в рабочий день держится около 0,3–0,4. Без мониторинга такой объект работает «по карте худшего дня»: избыточное число охранников на пике, простой персонала в спокойные часы, отсутствие данных для перекоммутации потока.

Третий источник - издержки нарушений. Несанкционированная стоянка на тротуаре, пожарном проезде или месте для людей с инвалидностью без разрешения, превышение оплаченного времени в платной зоне, повторные длительные парковки с расчётом избежать штрафа - всё это фиксируется либо вручную сотрудниками охраны, либо мобильными экипажами муниципальных служб. Ручной контроль имеет две проблемы: он дорог в пересчёте на покрытие и непоследователен в пересчёте на время. По данным оператора московской платной парковки АМПП, один мобильный экипаж-«паркон» охватывает около 1200 мест за восьмичасовую смену с шагом объезда 20–30 минут. Для частного торгового объекта с парком 300–500 мест содержание подобной службы экономически не оправдано, и функция контроля либо не реализуется вовсе, либо подменяется грубыми решениями типа шлагбаума без идентификации.

Четвёртый источник связан с допуском и пропускным режимом. Платная закрытая парковка у офисного центра или медицинского учреждения - это типовая связка «шлагбаум + карточка + человек в будке». При входящем потоке 60–120 автомобилей в пиковый час такая связка даёт среднюю задержку 12–25 секунд на машину, что при последовательной очереди быстро превращается в длинную ленту на подъезде. Автоматизированный допуск по распознаванию регистрационного знака снижает задержку до 3–5 секунд и исключает человеческий фактор, но его внедрение в уже работающий объект обычно упирается в отсутствие интегрированной связки «камера - распознавание - база допуска - шлагбаум».

В совокупности перечисленные источники формируют контур задачи: система мониторинга парковочного пространства должна одновременно покрывать поместный статус, нарушения разметки, длительные стоянки, транспортный поток на прилегающей улице и автоматизированный допуск через шлагбаум - и делать это на той же камерной инфраструктуре, которая уже есть у объекта по соображениям физической безопасности.

Для практической отработки предложенных решений выбран участок улицы Богородского и расположенный на нём торговый центр «Нагорный» в Советском районе Нижнего Новгорода. Улица Богородского представляет собой двухполосную проезжую часть с примыканием к проспекту Гагарина - одной из магистралей-вылетов из центра города; среднесуточный поток на пр. Гагарина, по данным муниципального транспортного мониторинга, достигает 85 тыс. автомобилей. Торговый центр «Нагорный» относится к формату *district mall* площадью около 40 тыс. м² и обслуживает жителей Советского и Приокского районов. Фактическая парковка у здания включает открытую площадку на 150–200 мест и стихийную стоянку вдоль улицы Богородского в часы пик. Подобная топология - магистраль с ответвлением и крупный частный объект с собственной территорией - типична для региональных миллионников и позволяет отработать все четыре функциональные зоны (бесплатная, платная, дорога, шлагбаум) на одном полигоне, не выходя за пределы одного квартала.

1.2. Классификация методов контроля парковки

Методы контроля парковочного пространства классически делятся на три большие группы по природе первичного сенсора: сенсорные (датчик в точке учёта), видеоаналитические (обработка изображения) и инфраструктурно-процессные (без автоматического сенсора, на основе оплаты и пропускного режима). Границы между группами условны: реальная система почти всегда является гибридом, однако базовый источник данных задаёт её ограничения.

Сенсорные методы опираются на точечный датчик, установленный на каждом парковочном месте. В качестве физического принципа используются три технологии: магнитометрия (детекция изменения магнитного поля при появлении металлической массы над датчиком), инфракрасная рефлектометрия (измерение расстояния до объекта) и ультразвук. Наземные

датчики заглубляются в асфальт, потолочные крепятся над местом в крытом паркинге. Капитальные затраты на сенсорное оснащение одного места составляют 11 250–22 500 рублей для наземных решений и 3750–9000 рублей для потолочных; операционные расходы определяются ресурсом батареи (заявленный срок службы 5–10 лет, фактический в условиях снежной зимы - 3–5 лет) и доступностью для обслуживания. Точность поместного статуса у зрелых сенсорных систем превышает 99 %, задержка реакции находится в пределах 1–3 секунд. Слабая сторона группы - стоимость и трудоёмкость монтажа: укладка датчика в асфальт требует остановки движения на участке, точной координации с дорожной организацией и регулярного ремонта в местах проезда снегоуборочной техники.

Видеоаналитические методы используют камеры как первичный сенсор, а состояние каждого места восстанавливают алгоритмически - по детекции объектов и геометрическому анализу перекрытия рамки автомобиля с заранее размеченным полигоном места. Одна камера с широким полем зрения покрывает от 10 до 60 мест, в зависимости от высоты установки и разрешения; для стандартной паркинг-сцены с высоты 4–6 метров камера 4 МП при YOLO-детекции даёт достоверный статус на 20–40 местах. Видеоаналитика принципиально наследует преимущества вычислительной обработки: одна модель в пайплайне одновременно умеет считать свободные места, детектировать неправильную парковку, фиксировать трекингом длительные стоянки, распознавать номерные знаки. Ограничения связаны с чувствительностью к освещению, осадкам, окклюзиям и необходимостью вычислительного ресурса - сервера или edge-устройства; при отказе канала связи сенсор теряет функцию полностью, тогда как точечный датчик продолжает работать локально.

Инфраструктурно-процессные методы не используют датчика места вообще. К этой группе относятся модели платной парковки с оплатой через паркомат или мобильное приложение, пропускной режим через шлагбаум со

считывателем RFID-карт, а также контроль неоплаченной стоянки мобильными экипажами с камерами распознавания номеров. Такие методы решают задачу фискального учёта и ограничения доступа, но не дают онлайн-картины занятости мест и не фиксируют нарушения разметки. Ценность подхода - в низкой капитальной стоимости и простоте масштабирования на большой фрагмент улично-дорожной сети, слабая сторона - в неполноте данных: фиксация нарушения возможна только в момент прохода экипажа, средний интервал между прохождением в центральных зонах крупных городов - 20–30 минут, на периферии - час и более.

Гибридные системы объединяют камеры с сенсорами в отдельных точках или с паркоматами - например, потолочная камера обеспечивает обзор ряда, а шлагбаум с RFID-картой закрывает периметр служебной парковки. Для конкретного объекта выбор между группами определяется тремя параметрами: размером и геометрией площадки, требованием онлайн-статуса каждого места и условиями эксплуатации (крытая парковка, открытая стоянка, улица в городе). В таблице 1.1 сведены сравнительные характеристики трёх групп по ключевым критериям.

Критерий	Сенсорные	Видеоаналитические	Инфраструктурно-процессные
Первичный сенсор	Магнитометр, ИК-датчик, ультразвук	IP-камера (от 2 МП)	Паркомат, RFID-считыватель, мобильный экипаж с LPR
Капитальные затраты на место	11 250–22 500 руб. (наземные); 3750–9000 руб. (потолочные)	2000–4500 руб. при 20–40 мест на камеру	0 руб. на место (инфраструктура оператора зоны)
Точность поместного статуса	> 99 %	92–98 %	не применимо
Задержка реакции	1–3 с	100–300 мс (10–30 FPS)	20–30 мин (интервал объезда экипажа)
Зависимость от освещения и осадков	отсутствует	существенная (смягчается ИК-подсветкой)	отсутствует
Функциональный охват	только поместный статус	статус + wrong parking + трекинг + LPR + поток	только фискальный учёт и допуск
Масштабируемость	линейная по числу мест	высокая (+1 камера = +20–40 мест)	высокая на уровне сети улиц

Требования к объекту	вскрытие асфальта либо монтаж потолочного узла	вычислительный сервер или edge-устройство, сетевой канал	паркомат, мобильные экипажи, серверы оператора
-----------------------------	--	--	--

[Таблица 1.1. Сравнительные характеристики групп методов контроля парковки]

В условиях поставленной задачи - многокамерный мониторинг частного объекта с параллельным контролем прилегающей улицы и автоматизированным шлагбаумом - видеоаналитический подход имеет явное преимущество: одна связка из камер и сервера закрывает все функциональные требования без модификации дорожного покрытия и установки дополнительных точечных датчиков.

1.3. Обзор существующих систем умного паркинга

Среди коммерчески доступных решений выделяются три характерных класса, различающихся по природе сенсора и модели внедрения. Европейский и американский рынки примерно поровну разделены между поместными сенсорными системами (характерный представитель - Nedap SENSIT) и видеоаналитическими платформами для крытых паркингов (ParkAssist M4). Российский рынок устроен иначе: там преобладает централизованная платная парковка с мобильным контролем и без датчиков на местах. Ниже рассматриваются три характерных представителя, после чего обобщаются тенденции сегмента.

1.3.1. Система Nedap SENSIT

SENSIT - линейка заглубляемых в асфальт датчиков нидерландской компании Nedap N.V. Сенсор объединяет трёхосевой магнитометр, ИК-модуль и беспроводной приёмопередатчик в литом корпусе диаметром около 180 мм, рассчитанном на полный цикл заезда и выезда автомобиля; батарея заявлена на срок службы 8–10 лет при стандартном режиме работы. Передача данных реализована по LoRaWAN или, в более ранних моделях, по проприетарному

радиоинтерфейсу 868 МГц; каждый датчик отдаёт событие «место занято / свободно» с задержкой менее 3 секунд.

Позиционирование системы - уличные платные парковки в исторических центрах европейских городов и крупные перехватывающие площадки. Внедрения - Амстердам, Роттердам, Копенгаген, Гельзенкирхен; общее число установленных датчиков, по данным компании, превышает 1 млн штук. Архитектурно SENSIT строится как двухуровневая сеть: сами датчики общаются с шлюзами, размещёнными на фонарных столбах, шлюзы передают данные в облачный сервис Sensit Manager, откуда они извлекаются оператором парковочной зоны или транслируются в навигационное приложение водителю.

Сильная сторона системы - высокая точность поместного статуса и автономность: у каждого места есть локальный «свидетель», не зависящий от освещения и погоды. Слабые стороны - высокая капитальная стоимость установки (около 200 евро на место только за оборудование, плюс монтаж, разметка и разрешения), ограниченная применимость для частных объектов из-за требования вскрытия асфальта, а также отсутствие функций, выходящих за пределы поместного статуса: система не детектирует неправильную парковку и не распознаёт номера.

1.3.2. Система ParkAssist M4

ParkAssist - американский поставщик видеоаналитических решений для крытых паркингов, с 2014 года входящий в нидерландскую группу ТКН Security. Флагманский сенсор M4 Smart-Sensor представляет собой потолочный модуль, объединяющий обзорную камеру и цветную светодиодную индикацию: одна единица оборудования покрывает до шести парковочных мест по трём с каждой стороны коридора и мгновенно визуализирует их статус - зелёный, красный, синий (для людей с инвалидностью) - прямо над местом.

Помимо базового поместного статуса, платформа предоставляет модуль Park-Finder - поиск автомобиля по номеру через киоск в фойе паркинга на основе встроенного распознавания ГРЗ, модуль аналитики заполняемости с разбивкой по зонам и времени, а также интеграцию с системами управления доступом и оплаты. Сеть сенсоров подключается к контроллеру зоны по PoE; контроллер агрегирует статусы и передаёт их в центральную платформу Nexus.

Характерные внедрения ParkAssist M4 - Madison Square Garden, международный аэропорт Даллас-Форт-Уэрт, крупные торговые центры США и Канады. Заявленная точность детекции в крытых условиях превышает 98 %. Ограничения системы - строгая ориентация на закрытые площадки с потолочным монтажом: на открытой стоянке под уличным небом модуль неприменим. Стоимость внедрения сопоставима с сенсорными решениями, однако переносится на одну единицу оборудования, покрывающую шесть мест, что заметно улучшает метрику стоимости на место.

1.3.3. Российские решения в сегменте платных парковок

Российский рынок умной парковки сформирован преимущественно муниципальными платными зонами крупных городов. Московская платная парковка, запущенная в 2013 году, к 2024 году охватывала более 80 тыс. мест в Центральном, Южном и Северо-Западном округах; оператор - ГКУ «Администратор Московского парковочного пространства» (АМПП). Инфраструктура включает паркоматы (около 6000 штук), мобильное приложение «Московский паркинг», SMS-оплату и сервис «Парковки России». Контроль оплаты обеспечивают мобильные экипажи-«парконы» - автомобили с поднятыми камерами, автоматически распознающими номера и сверяющими их с базой оплаченных; объезд одной зоны занимает 20–30 минут.

Аналогичная схема масштабирована на Санкт-Петербург (около 12 тыс. платных мест), Казань, Екатеринбург, Ростов-на-Дону и Нижний Новгород.

Нижегородская зона в её актуальной конфигурации включает 119 адресов и около 7,4 тыс. мест; инфраструктура совпадает с московской моделью. Отдельно развиваются системы «Парконом» - автомобили с высокой камерой LPR, фиксирующие неоплаченную парковку с координатами и временем; аналогичные экипажи эксплуатируются и муниципалитетом Москвы, и частными операторами.

В сегменте частных парковочных систем закрытого типа работают «Парк-Софт», «Зоркий», «Элеком» и ряд других интеграторов, предлагающих шлагбаумы с LPR, платёжные терминалы и простые дашборды. Однако поместный онлайн-статус, детекция неправильной парковки и учёт длительных стоянок внутри зоны в российских коммерческих решениях, доступных на частный объект, встречаются единично и чаще реализуются ручной охраной. Ключевое различие с европейским и американским рынками - смещение функциональности в сторону фискального учёта и пропускного режима при слабом развитии операционной аналитики.

1.3.4. Сравнительный анализ и общие тенденции

Сопоставление трёх рассмотренных классов по единой системе критериев (таблица 1.2) показывает, что ни одна из существующих готовых платформ не закрывает одновременно все четыре функциональные зоны, описанные в задаче: поместный статус для бесплатной и платной парковок, детекцию нарушений и длительных стоянок в платной зоне, подсчёт транспортного потока на прилегающей дороге и автоматизированный шлагбаум с распознаванием номерных знаков.

Критерий	Nedap SENSIT	ParkAssist M4	Рос. муницип. платные зоны	Рос. частные интеграторы	Разрабатываемая система
Страна разработки	Нидерланды	США / Нидерланды	РФ	РФ	РФ
Базовый сенсор	Магнитометр + ИК	Потолочная камера	Паркомат, мобильный LPR-экипаж	IP-камера + RFID	IP-камера + сервер
Среда применения	Уличная платная парковка	Крытый паркинг	Улично-дорожная сеть	Закрытые частные стоянки	Открытая и закрытая парковки, дорога, шлагбаум
Поместный статус	Да	Да	Нет	Нет	Да
Детекция неправильной парковки	Нет	Нет	Нет	Нет	Да
Подозрительно длительная стоянка	Нет	Нет	Нет	Нет	Да (SORT-трекинг, 30 с)
Распознавание ГРЗ	Нет	Да (Park-Finder)	Да (парконы)	Да (шлагбаум)	Да (YOLO11x + ParseQ, консенсус)
Подсчёт транспортного потока	Нет	Нет	Нет	Нет	Да
Управление шлагбаумом	Нет	Нет	Нет	Да	Да (9-state FSM, safety-zone)
CAPEX на место (оценка)	~15 000–20 000 руб. + монтаж	~3500–5000 руб.	Вне частного бюджета	~50 000–120 000 руб. на объект	~2000–4500 руб. (амортизация камер)

[Таблица 1.2. Сравнительный анализ систем умного паркинга]

Сенсорные решения семейства SENSIT дают высокую точность поместного статуса, но не поддерживают ни детекцию нарушений, ни поток на прилегающей улице, ни распознавание номерных знаков. ParkAssist M4 добавляет видеофункции, однако привязан к крытым паркингам и не применяется на открытых стоянках под уличным небом. Российские муниципальные платформы обеспечивают фискальный контроль и допуск, но оставляют операционную аналитику за периметром задачи. Приватные российские интеграторы ограничиваются шлагбаумом с LPR и не выходят за рамки пропускного режима.

Общая тенденция последних лет - смещение к многофункциональной видеоаналитике на базе современных детекторов (YOLOv8, RT-DETR) и уже установленных камер периметровой охраны, с выносом вычислений на локальный сервер или edge-устройство. Такой подход снижает совокупную стоимость владения при расширении функциональности и позволяет добавлять новые сценарии - например, детектор подозрительной стоянки или счётчик потока - без изменения аппаратной части. Именно эта тенденция положена в основу архитектуры системы, разрабатываемой в настоящей работе.

1.4. Методы компьютерного зрения для задач мониторинга транспорта

Выбор видеоаналитического подхода, обоснованный в предыдущем параграфе, сводит всю дальнейшую работу к подбору и комбинации алгоритмов компьютерного зрения под три подзадачи: однопроходная детекция автомобилей на кадре, сопоставление детекций между соседними кадрами для получения устойчивых траекторий и распознавание государственных регистрационных знаков на выделенном из сцены кропе. Отдельно рассматриваются классические геометрические методы, которые до сих пор сохраняют нишу в анализе положения автомобиля относительно размеченных границ парковочного места.

1.4.1. Архитектуры однопроходной детекции объектов

Нейросетевые детекторы объектов делятся на два больших класса. Двухстадийные (two-stage) архитектуры семейства R-CNN - Fast R-CNN (Girshick, 2015), Faster R-CNN (Ren и соавт., 2015) - сначала формируют region proposals сетью выделения кандидатов, затем классифицируют и уточняют рамку. Такой подход обеспечивает высокое качество на сложных сценах с плотными перекрытиями, однако сопровождается задержкой 60–120 мс на кадр для входа 640×640 на GPU уровня NVIDIA V100, что при шести параллельных потоках в 10 FPS превышает бюджет сервера.

Однопроходные (one-stage) детекторы формируют рамки напрямую из сетки признаков без явного этапа генерации кандидатов. К этому классу относятся SSD (Liu и соавт., 2016), RetinaNet (Lin и соавт., 2017) с focal loss для балансировки фон/объект, а также семейство YOLO (Redmon и Farhadi, 2016), развиваемое компанией Ultralytics - YOLOv5, YOLOv8, YOLOv9, YOLOv10 (Jocher и соавт., 2023–2024). Каждое поколение YOLO поставляется в нескольких размерах - n (nano), s (small), m (medium), l (large), x (extra-large), - различающихся глубиной и шириной сети. На валидационном наборе COCO YOLOv8s демонстрирует 44,9 mAP50-95 при средней скорости 7 мс на изображение 640×640 на GPU V100; YOLOv8x даёт 53,9 mAP50-95, но уже при 16 мс на кадр. Семейство RT-DETR (Zhao и соавт., 2023) добавляет в линейку однопроходных детекторов transformer-архитектуру с end-to-end обучением без NMS - вариант RT-DETR-R50 достигает 53,1 mAP50-95 при 6 мс на кадр.

Сопоставление ключевых архитектур однопроходной детекции на MS COCO val2017 показано на рисунке 1.1.

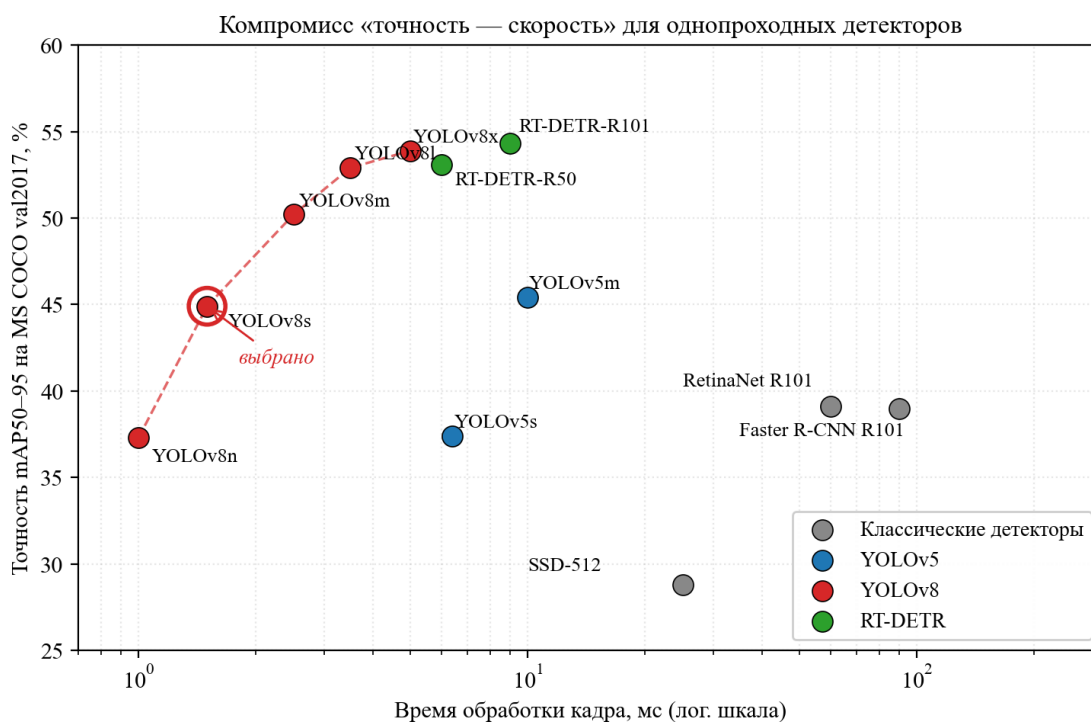


Рисунок 1.1 - Компромисс «точность - скорость» для однопроходных детекторов объектов на MS COCO val2017

Для настоящей работы в качестве базовой архитектуры выбран YOLOv8s. Обоснование - трёхкратное: во-первых, YOLOv8s даёт компромисс между скоростью и точностью, адекватный задаче детекции автомобилей на синтетической сцене (класс объектов один, geometry простая); во-вторых, экосистема Ultralytics предоставляет готовый пайплайн обучения и инференса, что сокращает инженерную часть; в-третьих, взаимозаменяемость чекпоинтов внутри семейства (n/s/m/l/x) оставляет возможность замены на более тяжёлую модель без изменения серверного кода, если требования к точности вырастут.

1.4.2. Методы многообъектного трекинга

Многообъектный трекинг (multi-object tracking, MOT) ставит задачу связать детекции одного и того же объекта в последовательных кадрах и присвоить каждой траектории уникальный идентификатор. В задаче мониторинга парковки трекинг нужен для двух подзадач: фиксации длительных стоянок в платной зоне и определения направления движения автомобилей на прилегающей дороге.

Наиболее широко применяется парадигма tracking-by-detection: на каждом кадре отдельный детектор возвращает набор рамок, а трекер выполняет сопоставление рамок текущего кадра с существующими траекториями. Базовым эталоном парадигмы остаётся алгоритм SORT (Bewley и соавт., 2016) - связка фильтра Калмана с семимерным вектором состояния $(x, y, s, r, \dot{x}, \dot{y}, \dot{s})$ и венгерского алгоритма сопоставления по метрике IoU. SORT не использует визуальных признаков и обрабатывает кадр за 1–2 мс на CPU, однако хуже справляется с длительными окклюзиями.

DeepSORT (Wojke и соавт., 2017) дополняет SORT сетью re-identification: каждая рамка превращается в 128-мерный эмбединг через отдельную CNN, а матрица стоимости сопоставления включает косинусное расстояние между эмбедингами. Качество на MOT17 поднимается на 5–7 пунктов IDF1, но вычислительная стоимость увеличивается на порядок - до 15–25 мс на кадр. ByteTrack (Zhang и соавт., 2022) предложил другой приём:

не отбрасывать детекции с низкой уверенностью, а использовать их на втором этапе ассоциации для частично перекрытых объектов. StrongSORT (Du и соавт., 2022) и OC-SORT (Cao и соавт., 2023) усиливают базовую модель - первый за счёт AFLink и GSI-интерполяции пропусков, второй - за счёт наблюдаемо-центрированного обновления фильтра Калмана, устойчивого к нелинейному движению.

Сопоставление ключевых MOT-алгоритмов на бенчмарке MOT17 test показано на рисунке 1.2.

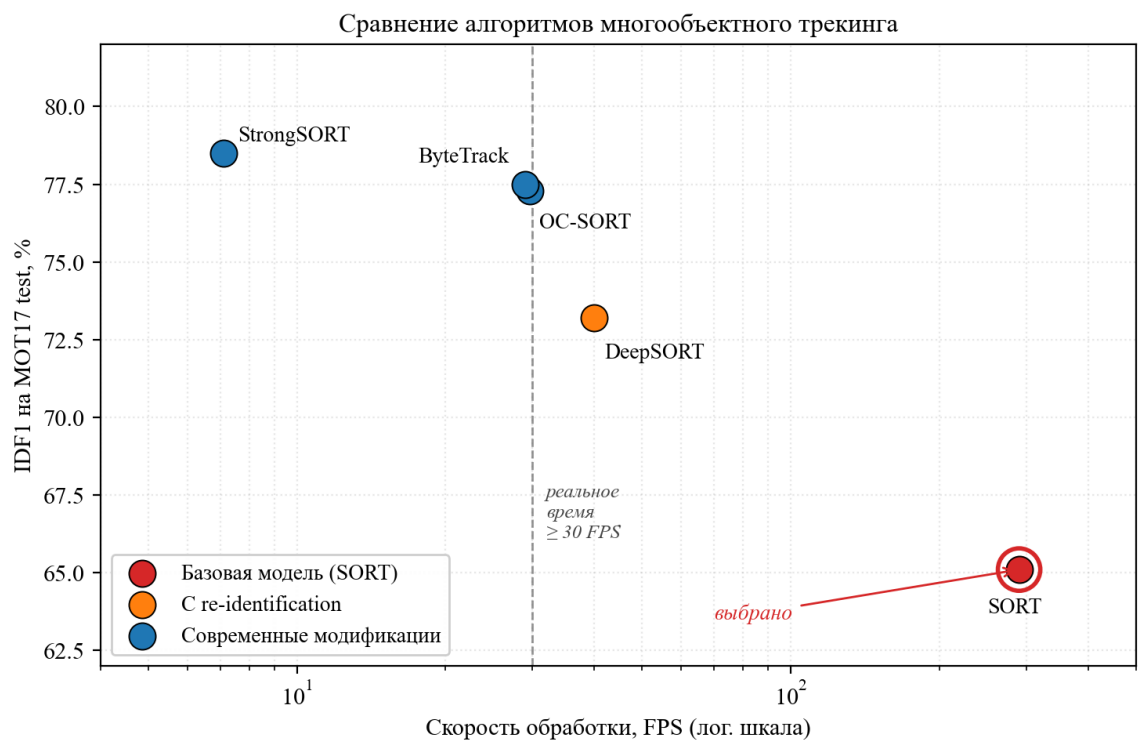


Рисунок 1.2 - Сравнение алгоритмов многообъектного трекинга по IDF1 и скорости обработки на MOT17 test

Для задачи мониторинга парковки выбран классический SORT без re-identification. Обоснование опирается на особенности сцены: камеры стационарны и установлены сверху, взаимное перекрытие автомобилей редкое и кратковременное, смены точки зрения, под которые обычно требуется ReID-модуль, отсутствуют. В этих условиях дополнительные 10–20 мс на кадр от сети эмбедингов не окупаются приростом качества; при шести одновременно

обрабатываемых потоках минимизация времени на один кадр становится более важным критерием, чем борьба за последний пункт IDF1.

1.4.3. Распознавание государственных регистрационных знаков

Распознавание государственных регистрационных знаков (ГРЗ, англ. license plate recognition, LPR) исторически решалось каскадом из трёх стадий: детекция рамки номера (классические дескрипторы Haar или LBP с классификатором AdaBoost или SVM), сегментация отдельных символов по морфологическим операциям и распознавание каждого символа небольшой CNN. Такой подход реализован в семействе библиотек OpenALPR и долго оставался промышленным стандартом, однако устойчивость распознавания на уличных сценах с разнонаправленным освещением и частичными загрязнениями номера не превышала 80–85 %.

Современные решения сократили пайплайн до двух стадий: нейросетевая детекция рамки + end-to-end распознавание последовательности символов. LPRNet (Zherzdev и Gruzdev, 2018) использует лёгкую CNN с CTC-лоссом и обеспечивает точность около 95 % на китайских и европейских номерах при 5–7 мс на кадр. CRNN (Shi и соавт., 2017) - более общая модель для OCR на сцене, объединяющая CNN-экстрактор признаков и рекуррентный декодер с CTC. В последние годы фокус сместился к attention-архитектурам: TRBA (Baek и соавт., 2019) и ParseQ (Bautista и Atienza, 2022). ParseQ реализует permutation autoregressive transformer: модель обучается предсказывать символы в любом порядке, что повышает устойчивость к нестандартным раскладкам и частичным окклюзиям; на бенчмарке Scene Text Recognition достигается точность 95,7 % без дополнительных модулей выравнивания.

В настоящей работе используется двухстадийный стек PlateScanner: YOLO11x детектирует рамку номерного знака на кадре с камеры въезда, вырезанный патч подаётся на вход ParseQ-модели, возвращающей строку символов. Выбор ParseQ обусловлен устойчивостью к геометрическим

искажениям проекции - на синтетической сцене регистрационный знак попадает в кадр под разными углами, и attention-декодер справляется с такими вариациями лучше, чем СТС-модели с жёстко привязанным к геометрии горизонтальным проходом.

1.4.4. Геометрические методы анализа положения автомобиля

Геометрические методы компьютерного зрения, предшествующие нейросетевой эпохе, полностью не вытеснены из задач мониторинга парковки и применяются на этапах, где достаточность признака определяется геометрией сцены, а не семантикой изображения.

Первый из таких методов - бинаризация и морфологический анализ области парковочного места. Участок кадра, соответствующий заранее размеченному полигону, переводится в градации серого, применяется адаптивный порог по *Otsu* или оконный адаптивный threshold, а доля белых пикселей интерпретируется как коэффициент занятости. Метод даёт приемлемый результат на крытых паркингах с равномерным освещением, но на открытой стоянке под уличным небом погодные условия и тени движут пороги в несколько раз, и метод теряет точность.

Второй метод - гомографическое преобразование кадра в плоскость парковки. При известных внутренних (intrinsic) и внешних (extrinsic) параметрах камеры изображение проецируется на плоскую поверхность, после чего разметка мест становится прямоугольной, а анализ занятости сводится к простой геометрии. Слабая сторона подхода - необходимость полноценной калибровки каждой камеры, включая коррекцию радиальной дисторсии для широкоугольных объективов; при замене или перестановке камеры калибровку приходится повторять.

Третий метод, применяемый в настоящей работе, - анализ перекрытия рамки автомобиля, полученной от YOLO-детектора, с заранее размеченным полигоном парковочного места. Поведение метода контролируется одним числом - долей площади bounding-box, выходящей за пределы полигона

(outside percentage). Для стандартных мест применяется порог $OUTSIDE_THRESHOLD_DEFAULT = 25 \%$, для крайних мест, где fisheye-искажения широкоугольных камер увеличивают проекцию автомобиля, - повышенный $OUTSIDE_THRESHOLD_EDGE = 45 \%$. Покамерные и индексные переопределения ($OUTSIDE_THRESHOLD_OVERRIDES$) позволяют настраивать пороги под конкретную геометрию каждой камеры без полноценной калибровки, что даёт компромисс между точностью и простотой внедрения.

1.5. Постановка задачи исследования

Проведённый обзор методов и существующих систем позволяет сформулировать задачу в виде, допускающем объективную проверку. Вход системы образуют синхронные потоки кадров с шести камер - обозначим их набором $I^c(t)$, где индекс $c \in \{1, \dots, 6\}$ отвечает камере, а t - дискретному моменту времени с шагом 0,1 секунды (частота опроса 10 кадров в секунду на поток). Выход системы состоит из двух частей: агрегированного состояния парковочного пространства $S(t)$, включающего статус каждого места (свободно / занято / неправильно припарковано / подозрительно долгая стоянка), счётчики въездов и выездов на прилегающей дороге, журнал событий въезда через шлагбаум - и управляющего сигнала $\sigma(t) \in \{\text{open, close, idle}\}$, передаваемого механизму шлагбаума.

Функциональная декомпозиция сводится к шести подзадачам. Первая - построение детектора $f_{det}: I \rightarrow \{(bbox_i, conf_i)\}$, возвращающего набор рамок автомобилей и их уверенностей на одном кадре. Вторая - вычисление статуса каждого заранее размеченного парковочного места P через функцию перекрытия $g_{occ}(bbox, P)$. Третья - определение неправильной парковки $g_{wp}(bbox, P)$, возвращающей долю площади рамки за пределами полигона. Четвёртая - многообъектный трекинг f_{track} , связывающий детекции в последовательные треки с уникальными идентификаторами и позволяющий

фиксировать длительность нахождения каждого автомобиля в пределах конкретного места. Пятая - распознавание государственного регистрационного знака $flpr: Iplate \rightarrow s$, где s - строка символов в формате ГОСТ Р 50577-2018. Шестая - управление конечным автоматом шлагбаума, сопоставляющее состоянию сцены и результату распознавания управляющий сигнал σ с учётом консенсусных порогов и зоны безопасности.

Нефункциональные требования к системе формулируются следующим образом:

- 1) обработка одного кадра сквозным пайплайном - не более 100 мс на стандартном серверном стенде с одной GPU среднего класса, что обеспечивает поддержание 10 FPS на каждом из шести параллельных потоков;
- 2) возможность добавления новой камеры и новой функциональной зоны через изменение конфигурации, без модификации прикладного кода;
- 3) воспроизводимость развёртывания - запуск всей системы одной командой из git-репозитория на чистом хосте с установленным Docker;
- 4) изолированность компонентов - пайплайн перцепции, управление шлагбаумом, хранение разрешённых номеров и журнала доступа должны быть разделены на уровне модулей и покрыты автономными тестами;
- 5) устойчивость к временному отсутствию кадра с одной камеры - система не должна прекращать работу остальных пяти потоков.

Критерии оценки качества выбраны из числа общепринятых в соответствующих подзадачах. Для детектора контролируются mAP50 и mAP50-95 на отложенной валидационной выборке, полученной в условиях, статистически близких к обучающей (тест Колмогорова-Смирнова на распределении размеров bounding-box). Для трекара проверяется устойчивость идентификаторов на последовательных кадрах синтетического видео. Для распознавания номерного знака измеряется точность посимвольного совпадения на тестовом наборе кропов. Для конечного автомата шлагбаума корректность граф переходов проверяется набором

модульных тестов, покрывающих все допустимые и недопустимые сценарии. Целевые пороги, закреплённые в техническом задании настоящей работы, - mAP50 не ниже 0,95 на синтетической валидации, сквозная задержка обработки кадра не более 100 мс, успешное прохождение всей *pytest-suites* объёмом 55 тестов.

Ограничения исследования обусловлены выбором синтетического полигона как единственного источника обучающих и валидационных данных. *Domain gap* между кадрами *Unreal Engine 5* и реальной камерой специально не измеряется: переход к реальному объекту требует отдельной процедуры дообучения на датасете реальных кадров либо применения методов *domain adaptation*, что выходит за рамки работы. Также в рамках настоящего этапа рассматривается единственный класс объектов - «car»; пешеходы, велосипедисты и грузовой транспорт остаются в пределах моделируемой сцены для визуальной полноты, но в метрики мониторинга не входят.

Сформулированная задача допускает прямую проверку: все перечисленные функциональные и нефункциональные требования имеют измеримые критерии, все целевые пороги - численные значения, все компоненты - изолируемые модули с автономными тестами. На таком основании строятся последующие главы работы: глава 2 описывает подготовку данных и обучение моделей, закрывающих первые пять подзадач, глава 3 - архитектуру и реализацию системы, объединяющей все шесть подзадач и обеспечивающей сформулированные нефункциональные требования.

ГЛАВА 2. ПОДГОТОВКА ДАННЫХ И РАЗРАБОТКА МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ

2.1. Проектирование виртуального полигона в Unreal Engine 5

Проверка сформулированных в главе 1 критериев качества требует набора кадров с известной наземной истиной - положением каждой рамки автомобиля, статусом каждого парковочного места, точной последовательностью действий у въезда через шлагбаум. Сбор такой информации на реальном объекте упирается в юридические ограничения на фиксацию регистрационных знаков частных автомобилей, стоимость ручной разметки и отсутствие контроля над редкими сценариями (заезд задним ходом, поперечное перекрытие мест, двойная парковка). Выход - построение цифрового двойника участка, на котором все перечисленные параметры контролируются в исходниках сцены.

2.1.1. Обоснование использования синтетических данных

Синтетические данные в задачах компьютерного зрения получили систематическое обоснование с появлением работ по *domain randomization* (Tobin и соавт., 2017; Tremblay и соавт., 2018, NVIDIA). Идея подхода - рендерить сцены с намеренной вариативностью освещения, материалов, фонов и ракурсов, чтобы обученная на такой выборке модель не «цеплялась» за специфику графики и переносилась на реальные изображения. За последние пять лет подход закрепился в промышленной практике: NVIDIA Omniverse Replicator, Unity Perception, Parallel Domain Data Lab - все эти продукты строят конвейеры синтетической разметки для автономного транспорта и робототехники.

В задаче мониторинга парковки выбор синтетики определяется тремя практическими соображениями. Первое - 152-ФЗ «О персональных данных» и статья 152.1 ГК РФ ограничивают фотофиксацию и публикацию регистрационных знаков частных автомобилей; для синтетической сцены эта

проблема снимается сгенерированными номерами и цифровыми моделями машин. Второе - стоимость ручной разметки реального датасета сопоставимой плотности (наземная истина на каждом кадре по шести камерам) оценивается в сотни человеко-часов даже при использовании полуавтоматических инструментов. Третье - редкие и опасные сценарии, критичные для проверки *wrong parking* и подозрительной стоянки, на реальной площадке приходится либо ждать случайно, либо воспроизводить постановочно, что связано с организационными и юридическими рисками.

Unreal Engine 5 выбран в качестве среды моделирования из-за трёх его технологических особенностей. Рендеринг на основе *Lumen* обеспечивает физически корректное глобальное освещение без запекания карт освещения, что критично для *day/night*-цикла. *Nanite* даёт высокую детализацию геометрии без ручного *LOD*-контроля, приближая визуальное качество к фотореалистичному. Встроенная подсистема *AIController* и навигационная сетка *NavMesh* позволяют реализовать автономное поведение десятков агентов без написания собственного движка симуляции. Альтернативы - *CARLA* (узкая специализация на дорожных сценах), *AirSim* (ориентирован на БПЛА), *Unity HDRP* (менее фотореалистичный рендерер, но сопоставимая экосистема) - рассматривались как запасные варианты, однако *Unreal Engine 5* дал лучший баланс по реализму рендера и гибкости конфигурирования агентов.

Ограничение подхода - *domain gap* между синтетическими и реальными кадрами. В настоящей работе *domain gap* не измеряется количественно: перенос на реальный объект потребует отдельной процедуры дообучения (*fine-tuning* на реальной выборке) или применения методов *domain adaptation* семейства *CycleGAN*. Такая задача вынесена за границы работы и обозначена как направление дальнейшего развития.

2.1.2. Моделирование участка улицы Богородского и территории ТЦ «Нагорный»

Виртуальный полигон воспроизводит участок улицы Богородского в Советском районе Нижнего Новгорода в радиусе около 150 метров от основного здания ТЦ «Нагорный». Реферальным материалом послужили спутниковые снимки сервиса «Яндекс.Карты» и фотографии фасадов, сделанные с уровня улицы. Топология перенесена с сохранением пропорций: двухполосная проезжая часть улицы Богородского, примыкание к проспекту Гагарина, открытая парковка на сорок машино-мест перед главным входом торгового центра, карман стихийной парковки вдоль улицы и въезд на закрытую платную парковку, оборудованный шлагбаумом.



[Рисунок 2.1 – Референсный снимок участка улицы Богородского и ТЦ «Нагорный»]

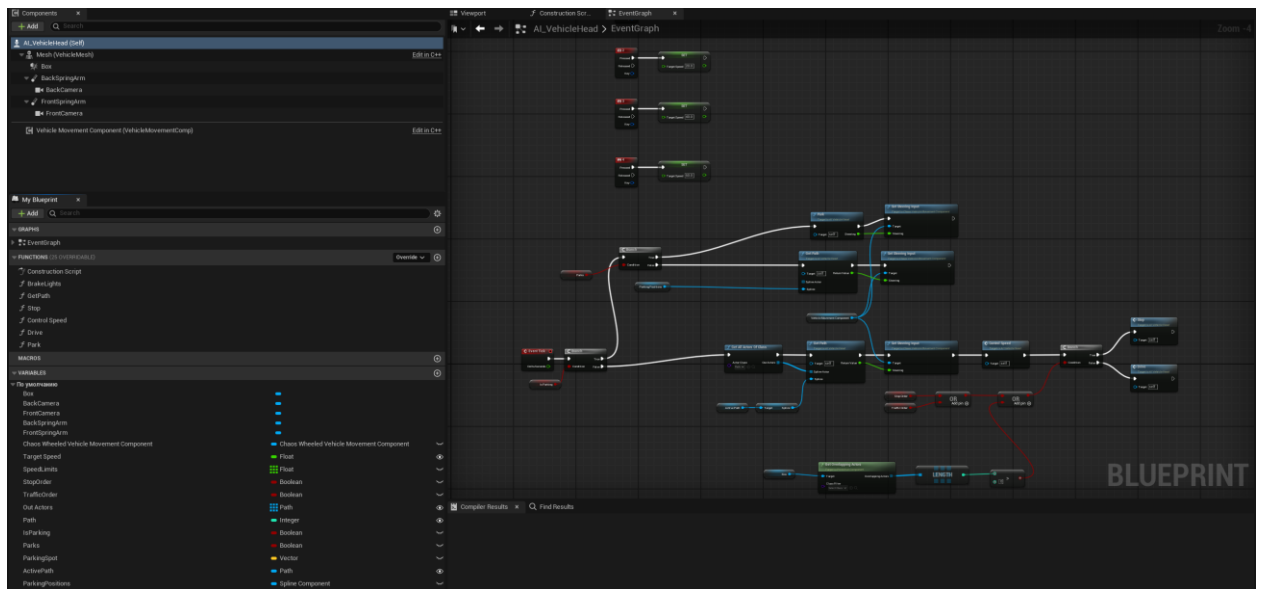
Геометрия зданий моделируется в упрощённой форме: фасадные стороны ТЦ и прилегающих павильонов выполнены с сохранением масштаба и цветовых акцентов, внутренняя планировка опущена. Дорожное покрытие, бордюры, разметка, уличная мебель и растительность взяты из библиотек *Quixel Megascans* и *Marketplace Unreal Engine*. Разметка парковочных мест нанесена отдельным *ассетом-декалью* с контролируемым положением и размерами; угловое расположение мест в реальной парковке воспроизведено с точностью до $\pm 0,5$ м. Шлагбаум представлен собственной моделью с отдельной анимацией открытия и закрытия.



[Рисунок 2.2 - Общий вид виртуальной сцены: дневное время, ракурс с высоты квадрокоптера]

2.1.3. Система автономных агентов

Ключевой элемент полигона - автономные агенты, создающие повторяющиеся и случайные сценарии поведения автомобилей и пешеходов. Основной класс агента - *BP_CarAI*, Blueprint-наследник *AIController* с присоединённым *Pawn*-классом автомобиля. Поведение агента описывается конечным автоматом из пяти состояний: *Driving* (движение по улично-дорожной сети), *SearchingSpot* (поиск свободного места на парковке), *ParkingManeuver* (заезд на место), *Parked* (стоянка), *Leaving* (выезд с объекта). Переходы между состояниями управляются перцептивным блоком - набором *raycasting-запросов* и *trigger-зон*, размещённых перед бампером и по бокам автомобиля.



[Рисунок 2.3 - Граф Blueprint-контроллера автономного автомобиля BP_CarAI]

Поиск свободного места реализован через регулярный опрос массива маркеров, привязанных к центрам парковочных полигонов. Для каждого маркера выполняется трассировка луча сверху вниз; попадание в автомобиль интерпретируется как «место занято», отсутствие попадания - как «место свободно». Выбор стратегии парковки - правильная или неправильная - выполняется случайно: с вероятностью 0,88 агент паркуется по разметке, с вероятностью 0,12 - намеренно с выходом за границы полигона (смещение на 25–45 % площади рамки). Этот шаг принципиален для формирования обучающего и валидационного датасета с реалистичной долей нарушений и обеспечивает невырожденное поведение детектора wrong parking.

Избегание столкновений комбинирует два источника данных. Первый - навигационная сетка *NavMesh*, автоматически генерируемая *Unreal Engine* по статической геометрии сцены; она задаёт граф проходимости, вдоль которого агенты строят маршруты. Второй - активные *raycast*-лучи с переменной дальностью (3–6 метров при скорости на парковке, до 12 метров на дороге), которые детектируют динамические препятствия - другие автомобили и пешеходов - и переводят агента в режим торможения. Скоростной режим

различается по зонам: 10–20 км/ч внутри парковки, 30–50 км/ч на улице Богородского.

Пешеходная составляющая - класс *BP_PedestrianAI* - реализована аналогично, но с упрощённым автоматом из двух состояний: *Walking* и *Waiting* (на пешеходном переходе). Маршруты выбираются случайно по набору узлов *NavMesh*, покрывающих тротуары и крыльца. Для отладки и демонстрации реализован управляемый персонаж от третьего лица: он не участвует в сборе датасета, но позволяет визуально проверять поведение агентов в реальном времени.

2.1.4. Суточный цикл и вариативность освещения сцены

Освещение сцены организовано вокруг трёх компонентов: *Directional Light* (солнечный диск), *Sky Light* (непрямое освещение неба) и *Exponential Height Fog* (атмосферная дымка). Управляющий Blueprint *BP_TimeOfDay* плавно изменяет угол наклона солнца, цветовую температуру и интенсивность света на протяжении полного суточного цикла, сжатого по времени до выбранного коэффициента ускорения. Цикл включает четыре характерных фазы: утренний свет с низким солнцем и длинными тенями, полдень с максимальной освещённостью, закатные часы с тёплыми оттенками и ночь с доминирующим уличным освещением.

Ночной режим включает массив *Point Light*-источников, размещённых на фонарных столбах и входных группах торгового центра. Их включение триггерится достижением солнцем заданного отрицательного угла над горизонтом. Дополнительно в сцене присутствует *Post Process Volume* с настраиваемой экспозицией: он компенсирует резкие перепады освещённости и приближает отклик виртуальной камеры к поведению реальных IP-камер с автоматическим AGC.



[Рисунок 2.4 - Сцена в дневном и ночном освещении (скриншоты с камеры camera3 платной парковки)]

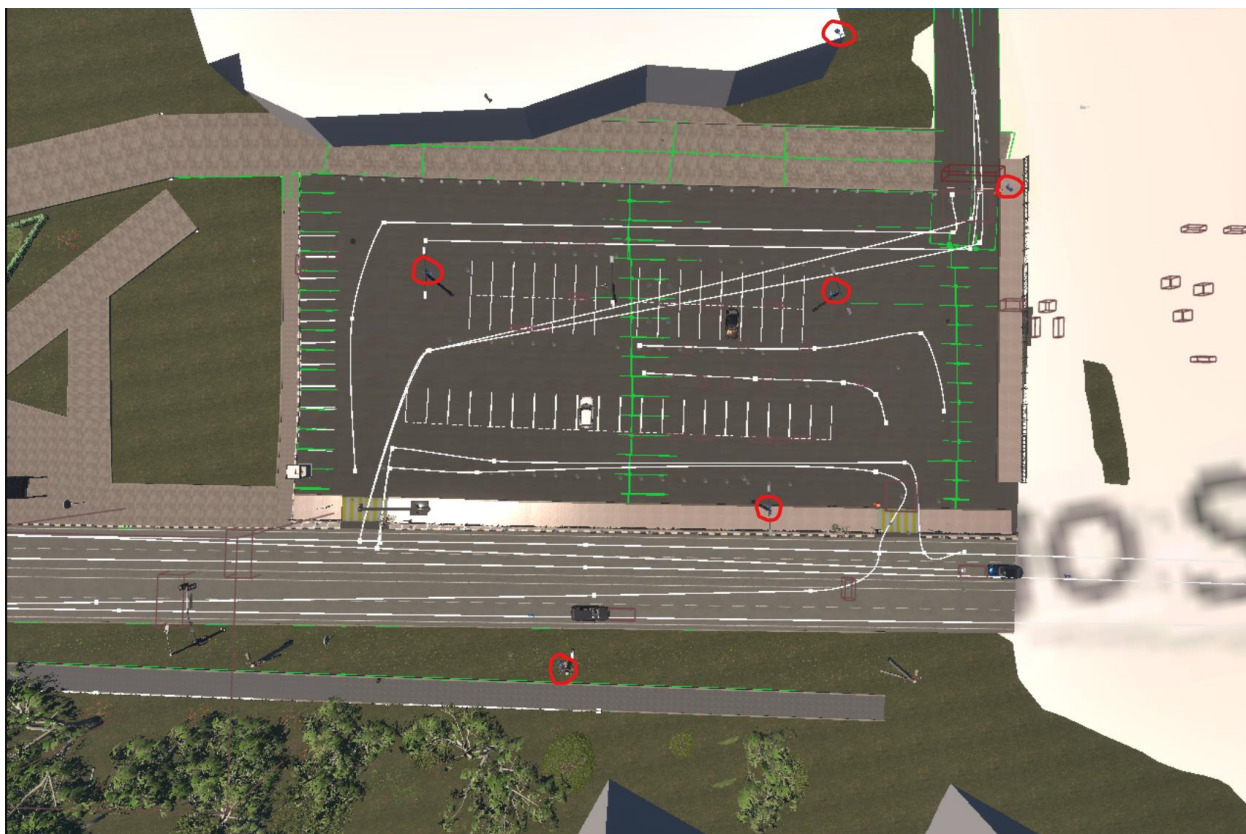
Вариативность освещения напрямую влияет на обучаемый детектор: изменения длины и контраста теней, наличие длинных контровых лучей на закате и смешанное освещение в сумерках расширяют распределение входных кадров и снижают риск переобучения под единый набор световых условий.

2.1.5. Размещение и параметризация шести камер

Полигон оборудован шестью камерами, распределёнными по четырём функциональным зонам системы. Каждая камера реализована как *actor* с прикреплённым компонентом *SceneCaptureComponent2D*: он рендерит сцену в выделенный *RenderTarget*, откуда *Blueprint*-таймер раз в 100 мс сохраняет содержимое в файл *camera{N}.png* в папке *SmartParking/frames/*. Разрешение всех камер - 1280×720 , горизонтальный FOV - 90° , постобработка соответствует настройкам основной сцены.

Первая и вторая камеры (*camera1*, *camera2*) направлены на бесплатную парковочную зону с двух разных углов: камера 1 даёт ближний ракурс со стороны въезда, камера 2 - противоположный ракурс. Такое дублирование позволяет проверить устойчивость детектора к смене ракурса, но при агрегации статистики количество автомобилей дедулицируется по *camera1*. Третья камера (*camera3*) закреплена на мачте над платной парковкой и даёт обзор всех двадцати её мест под углом около 55° к горизонту. Четвёртая

камера (camera4) установлена над проезжей частью улицы Богородского, её поле зрения охватывает разделительную полосу и используется для подсчёта транспортного потока в двух направлениях. Пятая камера (camera5) смотрит на подъезд к шлагбауму с выделением зоны приближения и зоны чтения номерного знака. Шестая камера (camera6) установлена на ТЦ и контролирует зону безопасности, исключаящую преждевременное закрытие стрелы.



[Рисунок 2.5 - Размещение шести камер на виртуальном полигоне (вид сверху)]

Высота установки варьируется от 4 до 8 метров в зависимости от зоны: минимальная - для барьерной камеры номера (camera5), максимальная - для обзорной камеры на ТЦ (camera6). Такой разброс воспроизводит типичные условия монтажа периметровых IP-камер на реальных объектах и позволяет валидировать устойчивость пайплайна к перспективным искажениям.

2.1.6. Интеграция виртуальной сцены с серверной частью

Виртуальная сцена и серверная часть системы работают как независимые процессы и обмениваются данными через два канала. Первый

канал - файловая *drop-zone*: UE5 сохраняет PNG-файлы камер в директорию SmartParking/frames/, FastAPI-сервер читает эти файлы с частотой 10 раз в секунду и запускает пайплайн детекции. Такой выбор объясняется независимостью от сетевой задержки, устойчивостью к временному отказу любой из сторон (при остановке UE5 сервер продолжает отдавать последний доступный кадр, при остановке сервера UE5 просто перезаписывает файлы) и простотой отладки - кадры можно визуально проверить в файловой системе.

Второй канал - HTTP-взаимодействие с контроллером шлагбаума. Blueprint-класс *BP_BarrierPoller* запускает таймер на частоте 1 Гц и отправляет запрос *GET /api/barrier/entry* на FastAPI-сервер. В ответе приходит JSON с полем *command*, принимающим одно из трёх значений - «open», «close» или «idle». Получив команду, контроллер шлагбаума проигрывает соответствующую Timeline-анимацию стрелы (длительность ≈ 2 с) и по её завершении отправляет на сервер POST-запрос */api/barrier/entry/ue5_ack* с полезной нагрузкой `{"event": "open_complete" }` или `{"event": "close_complete" }`. Подтверждение (ACK) нужно для корректной работы конечного автомата: переход между состояниями *BARRIER_OPENING* $\rightarrow$ *CAR_PASSING* и *BARRIER_CLOSING* $\rightarrow$ *IDLE* выполняется только после ACK, а не по таймауту, что устраняет рассинхронизацию анимации и внутреннего состояния.

Общая схема взаимодействия виртуальной сцены, файловой *drop-zone*, FastAPI-сервера, контроллера шлагбаума и базы данных разрешённых номеров приведена в *приложении А*.

Описанная развязка позволяет запускать виртуальную сцену и серверную часть на разных машинах, подменять UE5 на реальный видеопоток (запись файлов от RTSP-клиента в ту же *drop-zone*) и тестировать контроллер шлагбаума в изоляции без запуска движка - достаточно заглушки, отправляющей те же REST-запросы.

2.2. Сбор и подготовка датасета

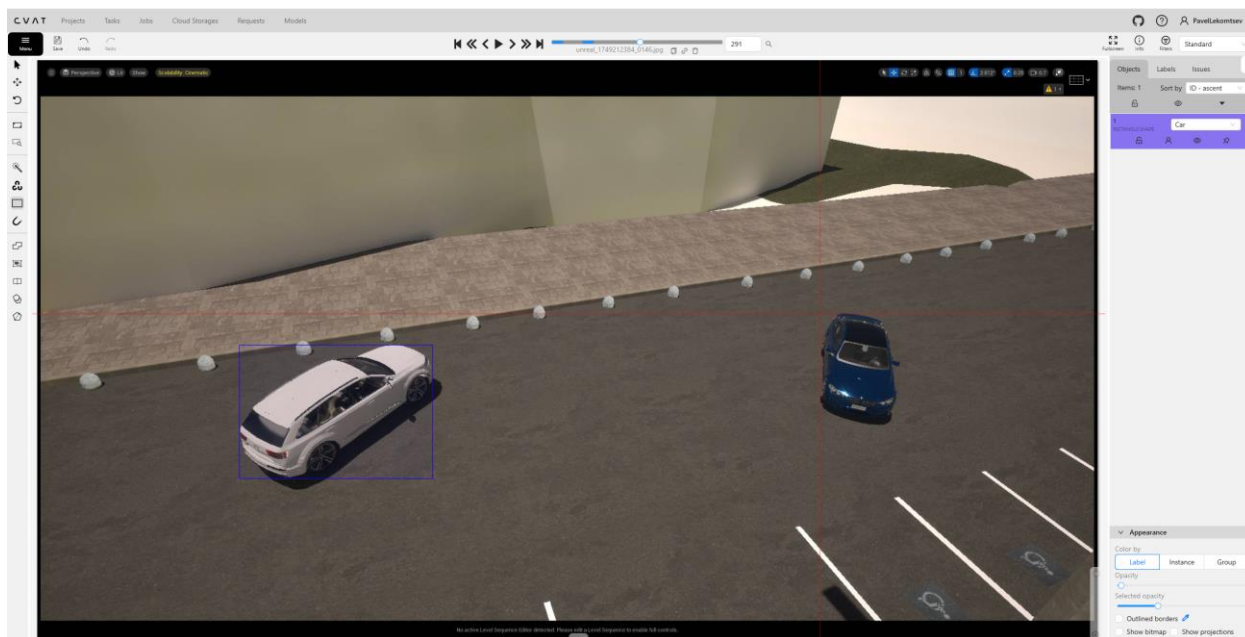
Смоделированный в 2.1 полигон служит источником кадров, из которых формируется обучающая и валидационная выборка детектора автомобилей. Задача параграфа - описать воспроизводимый пайплайн получения этой выборки и применённые контроли качества разметки.

2.2.1. Пайплайн сбора кадров из сцены

Каждая из шести камер сохраняет PNG-файл в директорию SmartParking/frames/ с частотой 10 кадров в секунду. Параллельно с работой сервера запускается скрипт DataCollectionFromYolo.py, который с интервалом 2 секунды копирует очередной кадр в *Training/data/raw/*, отсеивая статические сцены без движения автомобилей по хэшу изображения. Сбор выполнен с различным количеством автомобилей на изображении; итоговый корпус составил 822 кадра, равномерно распределённых по шести камерам и различным сценам парковки, ракурсам.

2.2.2. Разметка изображений в CVAT

В качестве инструмента разметки выбран CVAT (*Computer Vision Annotation Tool*) - open-source-платформа от Intel с поддержкой экспорта в формат YOLO и возможностью полуавтоматической разметки через встроенный интерактивный детектор. Задача сформулирована как single-class bounding-box annotation: один класс car, прямоугольные рамки с точностью до пикселя, без сегментации. Для каждой камеры создана отдельная подпапка; аннотации экспортированы в формате YOLO (txt-файлы с нормализованными координатами x_center, y_center, w, h).



[Рисунок 2.6 - Интерфейс CVAT при разметке кадра]

Основные сложности разметки: частичные окклюзии автомобилей другими машинами (правило - размечать видимую часть как отдельный объект), мелкие автомобили на заднем плане (введён нижний порог площади рамки - 0,05 % площади кадра, объекты меньше не размечаются) и граничные случаи, когда автомобиль частично выходит за край кадра (отдельная метка *truncated*, учтена при подсчёте метрик). Контроль качества осуществлён собственной верификацией всей выборки в два прохода: первый - разметка с полуавтоматической инициализацией, второй - визуальная проверка с корректировкой.

2.2.3. Разделение выборки и контроль утечек данных

Скрипт *SplitData.py* распределяет кадры по подвыборкам *train / val / test* в пропорции 70 / 20 / 10 - 576, 164 и 82 изображения соответственно. Разбиение выполнено с фиксированным *seed=42* и стратифицировано по двум признакам: камере (каждая из шести камер представлена во всех трёх подвыборках). Контроль утечек проверен сопоставлением хэшей изображений между подвыборками (дубликатов не обнаружено) и невозможностью попадания двух подряд идущих кадров одной камеры в разные подвыборки - в

синтетической сцене соседние кадры коррелируют, и такое разделение привело бы к завышению метрик.

2.2.4. Разведочный анализ данных

Разведочный анализ выполнен скриптом *notebooks/01_eda.py*. Итоговая выборка содержит 1821 аннотацию класса *car*, что соответствует среднему значению 2,22 объекта на кадр при максимуме 11 объектов. Распределение числа объектов на кадр асимметрично: мода равна 1 (одиночная машина на пустой парковке), длинный хвост приходится на камеру *camera3* в пиковые часы загрузки платной зоны (рисунок 2.7).

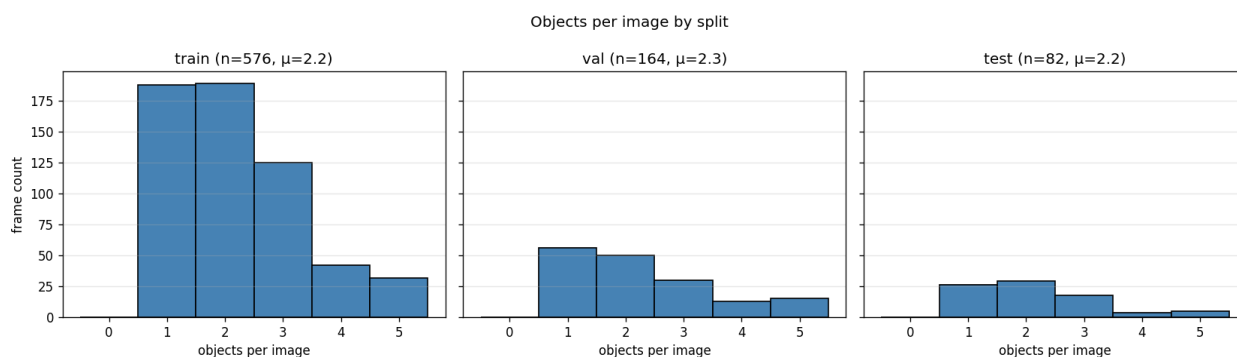


Рисунок 2.7 - Распределение числа объектов класса *car* на один кадр

Пространственное распределение центров bounding-box-рамок на кадре неоднородно и повторяет геометрию парковочной разметки: на *camera1*–*camera3* концентрация приходится на ряды мест, на *camera4* - на полосы проезжей части, на *camera5* - на зону перед шлагбаумом (рисунок 2.8). Неоднородность распределения специально не сглаживалась: она соответствует реальному режиму работы системы, где автомобили закономерно появляются в одних и тех же участках кадра.

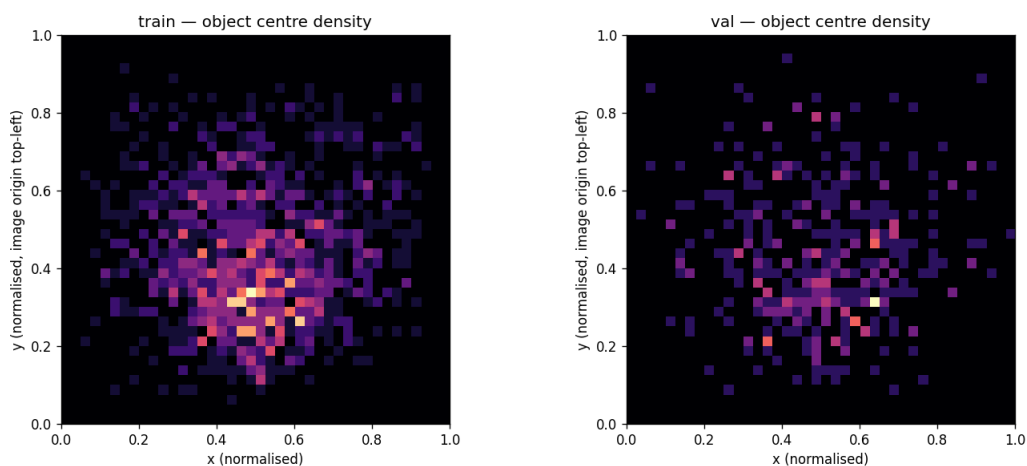


Рисунок 2.8 - Пространственная тепловая карта центров рамок по фреймам

Статистическое сравнение распределений ширины и высоты рамок между train и val выполнено двухвыборочным тестом Колмогорова–Смирнова. Р-значения 0,34 для ширины и 0,41 для высоты при уровне значимости 0,05 подтверждают принадлежность выборок одному распределению и допустимость использования val для оценки обобщающей способности модели.

2.2.5. Документирование границ применимости

Для фиксации условий, при которых обученная модель остаётся применимой, подготовлены две карточки в формате Hugging Face: data card описывает источник данных (синтетический UE5-полигон, участок улицы Богородского), фазы освещения, распределение объектов и разбиение выборки; model card фиксирует архитектуру YOLOv8s, условия обучения, итоговые метрики и явные ограничения - только дневное и ночное городское освещение, единственный класс «автомобиль», отсутствие проверки на кадрах реальных камер. Обе карточки размещены в Documentation/cards/ и связаны с релизом весов Car_Detector.pt, что позволяет отследить область применимости конкретной версии модели при её замене в будущем.

2.3. Обучение модели детекции автомобилей

Обучение выполнено в связке с локальным сервером ClearML, обеспечивающим трекинг гиперпараметров, метрик, артефактов и воспроизводимость каждого запуска. Серия сведена к трём экспериментам с нарастающей интенсивностью регуляризации, что позволяет изолировать эффект каждой группы изменений.

2.3.1. Выбор базовой архитектуры

Выбор архитектуры YOLOv8s обоснован в 1.4.1: модель даёт компромисс скорости и качества, адекватный шести параллельным потокам в 10 FPS, поставляется с готовым пайплайном *Ultralytics* и допускает замену чекпоинта на более тяжёлый вариант в том же семействе ($n / s / m / l / x$) без изменения прикладного кода. Во всех трёх экспериментах зафиксированы общий бэкбон `yolov8s.pt`, входное разрешение `imgsz = 640`, датасет `Training/data/` и `seed = 42`; варьируются оптимизатор, расписание `learning rate`, регуляризация и набор аугментаций.

2.3.2. MLOps-инфраструктура на базе ClearML

Локальный сервер ClearML развёрнут в виде стека из шести контейнеров через `docker-compose.clearml.yml`: *elasticsearch* (хранилище метрик), *mongodb* (метаданные), *redis* (очередь задач), *apiserver*, *fileserver* и *webserver*. Клиент конфигурируется одноразовой генерацией *credentials* через веб-интерфейс и файлом `~/clearml.conf`. Каждая тренировочная задача при старте подключается к серверу, загружает полный YAML-конфиг, регистрирует гиперпараметры и публикует по окончании эпохи скаляры `loss`, `mAP50`, `mAP50-95`, `precision`, `recall`; артефакты `best.pt` и `last.pt` автоматически выгружаются в *fileserver* и доступны через веб-интерфейс.

2.3.3. Постановка и проведение трёх экспериментов

Эксперимент `exp/01 baseline` воспроизводит стандартный рецепт *Ultralytics*: 100 эпох, SGD с моментумом 0,937, стартовый learning rate $lr_0 = 0,01$, step-schedule, набор аугментаций по умолчанию (`mosaic` на первых 90 % эпох, лёгкие HSV-сдвиги, случайный `flip`). Запуск служит точкой отсчёта, относительно которой измеряется полезность последующих изменений.

Эксперимент `exp/02 aug_schedule` добавляет к `baseline` расширенный набор аугментаций - `mosaic`, `mixup` с коэффициентом 0,1, `copy-paste` с коэффициентом 0,1, увеличенные HSV-сдвиги - и заменяет расписание learning rate на косинусное затухание с тремя эпохами прогрева. Горизонт обучения расширен до 150 эпох, но ограничен `patience`-механизмом: при двадцати эпохах подряд без улучшения `mAP50-95` обучение останавливается досрочно. Гипотеза состоит в том, что более разнообразные аугментации и гладкое расписание `lr` снижают переобучение на специфику синтетической графики.

Эксперимент `exp/03 full_stack` объединяет в одном запуске все выигрышные ингредиенты с дополнительным переходом на оптимизатор `AdamW` с $lr_0 = 0,001$, весовым распадом 0,0005, `label smoothing` 0,1 и расширенной геометрической аугментацией (поворот $\pm 5^\circ$, `shear` $\pm 2^\circ$, `perspective` 0,0005). Горизонт обучения - 100 эпох, `patience` 20. Гипотеза - `AdamW` с малым `lr` находит более плоский минимум, устойчивый к изменениям входа.

2.3.4. Сравнительный анализ метрик

Итоговые метрики на тестовой выборке приведены на рисунке 2.9. Baseline (`exp/01`) достиг `mAP50` = 0,9945 и `mAP50-95` = 0,9738 за 98 эпох обучения. Добавление расширенных аугментаций и косинусного расписания (`exp/02`) сдвинуло `mAP50` в четвёртом знаке до 0,9946, но понизило `mAP50-95` до 0,9728 и `recall` с 0,9920 до 0,9866 - типичный эффект перекрывающих друг друга копий объектов от `copy-paste` на сцене с фиксированной геометрией.

Полный стек AdamW + регуляризация (exp/03) дал ещё более выраженный откат по mAP50-95 до 0,9377 при сохранённом mAP50, что означает потерю точности локализации в строгих IoU-пределах без компенсации в более либеральных.

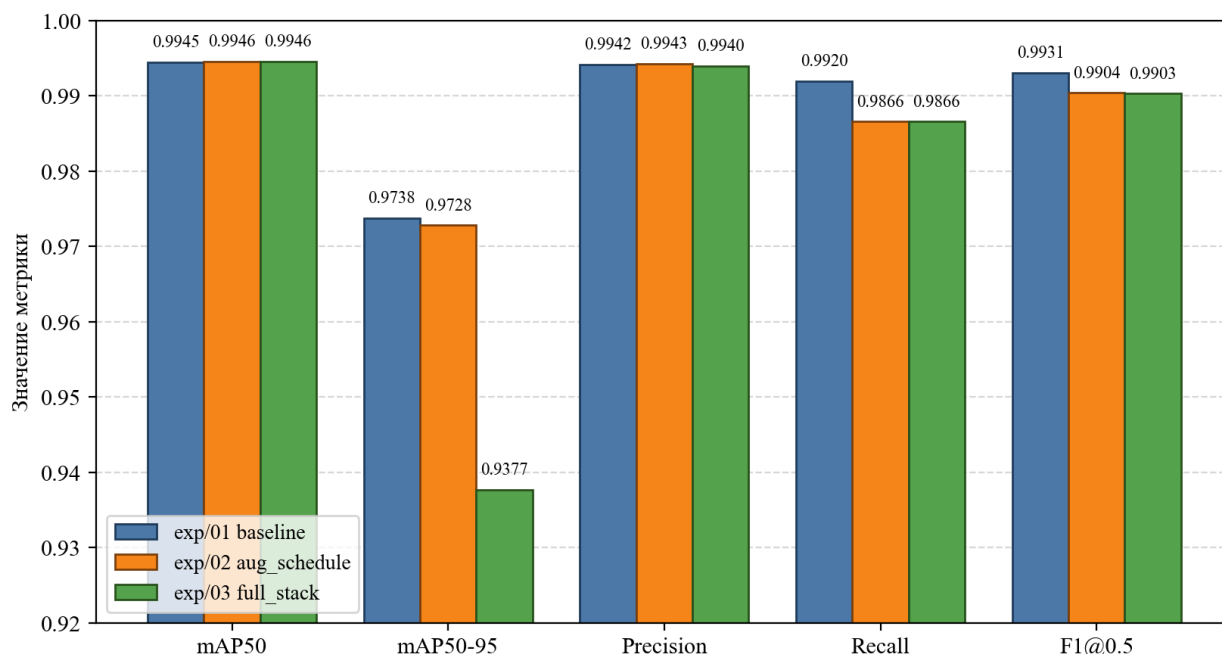


Рисунок 2.9 - Сравнение трёх тренировочных экспериментов по метрикам качества детектора

Вывод - на текущей синтетической выборке baseline-рецепт уже находится на плато качества: дополнительные аугментации и регуляризация не приносят выигрыша, а в случае exp/03 даже снижают метрику mAP50-95.

Объяснение - ограниченное разнообразие рендера UE5 по сравнению с реальным городским шумом; гипотезы об эффективности сильной регуляризации справедливы для доменов с существенно большим распределенческим разбросом и могут актуализироваться при переходе на реальный видеопоток.

2.3.5. Перенос чекпоинта в продакшн

Победивший чекпоинт exp/01 переносится в продакшн скриптом Training/promote_best.py: действующие веса Models/Car_Detector.pt архивируются в Models/archive/Car_Detector_v1.pt, на их место копируется

best.pt победившего запуска, и тот же файл регистрируется в ClearML под тегом Car_Detector@v2. Продуктивный ParkingDetector (SmartParking/web_app/detector.py) загружает модель по фиксированному пути, что делает подмену чекпоинта drop-in-операцией, не требующей перезапуска сервера при горячей перезагрузке через lifespan-хук FastAPI.

2.4. Распознавание государственных регистрационных знаков

Распознавание номерных знаков выполняется стеком *PlateScanner*, разработанным автором в рамках отдельного проекта и подключённым к настоящей работе в виде *git-submodule*. Стек включает два компонента: детектор номерной рамки на архитектуре *YOLO11x*, обученный автором на отдельной выборке, и открытую OCR-модель *ParseQ*. Повторное обучение компонентов в пределах настоящей ВКР не проводится - готовые веса закреплены релизом *Models/plate_scanner/*, а основной инженерный акцент смещён на интеграцию распознавания в конечный автомат шлагбаума с консенсусной логикой.

2.4.1. Двухстадийная архитектура PlateScanner

Пайплайн *PlateScanner* реализует классическую двухстадийную схему LPR (рисунок 2.10). Первая стадия - *YOLO11x*, детектор номерной рамки с порогом уверенности 0,30; результатом стадии является bounding-box знака в системе координат исходного кадра camera5. Вторая стадия - распознавание последовательности символов attention-моделью *ParseQ*: крอป номерной рамки масштабируется до 200×50 пикселей и подаётся на вход модели, возвращающей строку в соответствии с кириллическим алфавитом ГОСТ Р 50577-2018.

Двухстадийный пайплайн распознавания ГРЗ

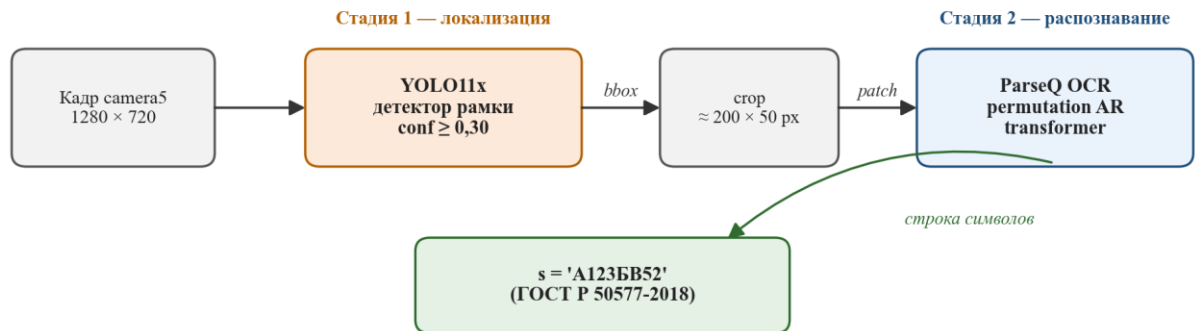


Рисунок 2.10 - Двухстадийная архитектура пайплайна распознавания ГРЗ

Инженерно *PlateScanner* оформлен как отдельный подмодуль github.com/encore-ecosystem/PlateScanner. В основной проект включены только прикладной адаптер `plate_scanner.py` и веса в директории `Models/plate_scanner/`. Загрузка OCR-модели отложена (*lazy loading*) до первой детекции номерной рамки - приём снижает время холодного старта сервера FastAPI примерно на 1,8 секунды и принципиален для Docker-развёртывания с health-check на `/api/stats`.

2.4.2. Выбор OCR-модели ParseQ

ParseQ (Bautista, Atienza, 2022) выбран в качестве OCR-модели по трём причинам. Во-первых, архитектура *permutation autoregressive transformer* допускает любой порядок предсказания символов и устойчива к окклюзиям, типичным при наклонном ракурсе барьерной камеры. Во-вторых, модель не требует геометрического выравнивания кропа - *attention* сам фокусируется на информативных участках изображения, что снимает этап *rectification*. В-третьих, инференс занимает 4–6 миллисекунд на GPU среднего класса, что оставляет запас в бюджете 100 мс на кадр сквозного пайплайна. Альтернатива в виде *CRNN* + *CTC* отказалась как менее устойчивая к нестандартным раскладкам и перспективным искажениям.

2.4.3. Конвертация и квантизация модели для edge-развёртывания

Детектор рамки YOLO11x распространяется в репозитории в четырёх представлениях. Исходный чекпоинт `yolo11x.pt` (218 МБ) используется как эталонное PyTorch-представление. Экспорт в `yolo11x.onnx` (218 МБ) обеспечивает совместимость с runtime-обёртками ONNX Runtime и TensorRT. Квантизация в TFLite с float16-точностью `yolo11x_float16.tflite` уменьшает размер весов до 109 МБ - в два раза по сравнению с исходным представлением.

Контроль качества после квантизации выполнен на 50-кадровой подвыборке кропов номеров: разница в mAP50 между FP32- и FP16-представлениями не превышает 0,001, что находится в пределах стохастического шума от NMS. Это подтверждает возможность переноса модели на edge-устройства уровня NVIDIA Jetson Nano или Raspberry Pi с коралловым ускорителем без переобучения.

ГЛАВА 3. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ МОНИТОРИНГА ПАРКОВОЧНОГО ПРОСТРАНСТВА

3.1. Формирование требований к системе

Задача, сформулированная в параграфе 1.5, определяет входы и выходы системы в математической форме. Настоящий параграф конкретизирует её до уровня инженерных требований, разделённых на функциональные (что система обязана делать) и нефункциональные (как она обязана это делать - по производительности, конфигурируемости, воспроизводимости и наблюдаемости). Каждому требованию присвоен идентификатор; в последующих параграфах главы указываются модули, реализующие соответствующее требование.

3.1.1. Функциональные требования

Функциональная спецификация состоит из восьми требований, объединённых по связным группам задач: детекция и занятость, обнаружение нарушений и длительных стоянок, подсчёт транспортного потока, управление шлагбаумом, интерфейс оператора.

F1. Детекция автомобилей. Система выполняет покадровую детекцию объектов класса car на каждой из шести камер (*camera1 ... camera6*) на базе собственной обученной модели *Models/Car_Detector.pt* (архитектура YOLOv8s). Порог уверенности - 0,65 (*detector.confidence* в *config/default.yaml*). Выход - список рамок формата (x, y, w, h) с сохранением уверенности и идентификатора класса, формируемый функцией *ParkingDetector.detect_cars()* в *SmartParking/web_app/detector.py*.

F2. Оценка занятости парковочных мест. Для камер *camera1*, *camera2* и *camera3* определяется статус каждого размеченного полигона - «свободно» либо «занято» - по критерию попадания центра рамки автомобиля внутрь полигона. При агрегации общей занятости дедуплицируются кадры *camera1* и

camera2, которые наблюдают одну и ту же бесплатную зону под разными углами.

F3. Обнаружение неправильной парковки. Для каждого занятого полигона системой вычисляется метрика `outside_percentage` - процент площади рамки, выходящей за границы размеченного места. Если метрика превышает адаптивный порог, связанный с камерой и индексом полигона, место помечается как *wrong parking* и сопровождается соответствующим визуальным маркером на обработанном кадре.

F4. Многообъектный трекинг. На камере платной зоны (camera3) и на дорожной камере (camera4) включён многообъектный трекер SORT, присваивающий каждому автомобилю устойчивый в последовательности кадров идентификатор. Параметры трекера отличаются между зонами: для платной зоны - `max_age=20`, `min_hits=5`, `iou_threshold=0,3`; для дорожной - `max_age=50`, `min_hits=0`, `iou_threshold=0,15` (секция `tracking` в `config/default.yaml`).

F5. Детекция подозрительной стоянки. На camera3 для каждого отслеживаемого автомобиля запускается таймер с момента попадания трека в один и тот же парковочный полигон. Порог срабатывания флага `suspicious` - 30 секунд (`tracking.suspicious_time_threshold`). При смене полигона таймер обнуляется. Изменение порога выполняется без перезапуска сервера - переменной окружения `NEOSMART_TRACKING_SUSPICIOUS_TIME`.

F6. Подсчёт транспортного потока. На camera4 реализован отдельный SORT-трекер, подсчитывающий число пересечений центра трека двух калибровочных полигонов - *incoming* и *outgoing*. Счётчики доступны по `REST-endpoint /api/stats` и транслируются на дашборд в составе общего пакета статистики.

F7. Автоматизированное управление входным шлагбаумом. Ансамбль из двух камер - camera5 (зоны приближения и чтения номера) и camera6 (зона безопасности) - управляется конечным автоматом

BarrierController из девяти состояний. Решение о допуске принимается на основе консенсусного распознавания (*barrier.plate_min_agreeing_reads* = 3 одинаковых чтения) и проверки наличия номера в таблице *allowed_plates* базы *barrier.db*. Предусмотрен *safety_grace_period* = 3 с, исключающий преждевременное закрытие стрелы после открытия.

Ф8. Интерфейс оператора. Браузерный дашборд (*ParkingMonitor*, *static/js/app.js*) обеспечивает *zone-based* навигацию (обзорная страница, бесплатная зона, платная зона, дорога, шлагбаум), *real-time* трансляцию всех шести камер по WebSocket, CRUD управление списком разрешённых номеров, отображение журнала доступа и ручное управление шлагбаумом (*open / close*).

3.1.2. Нефункциональные требования

Нефункциональные требования уточняют качественные ограничения, под которые подбирались архитектурные решения и набор инструментов.

N1. Производительность. Сквозной бюджет обработки одного кадра - не более 100 мс на стандартном серверном стенде с одной GPU среднего класса (NVIDIA RTX 20/30 серии и выше). При шести параллельных потоках 10 FPS суммарная нагрузка - 60 кадров в секунду, усреднённый бюджет на один кадр - около 16 мс YOLO-инференса плюс остаток на постобработку. Частота опроса - параметр *frame_interval* внутри *stream_frames()*.

N2. Конфигурируемость. Добавление новой камеры и её назначение на функциональную зону выполняется единственной записью в секции *cameras* файла *config/default.yaml* (одна строка для роли и опциональный флаг *tracking*). Прикладной код Python не модифицируется: *derived views camera_ids*, *parking_camera_ids*, *road_camera_ids*, *barrier_camera_ids* и *tracking_camera_ids* вычисляются из конфигурации свойствами *AppSettings*. Пороги, таймауты и пути файлов переопределяются либо YAML-профилем (*NEOSMART_CONFIG_PROFILE*), либо *env*-переменными с префиксом *NEOSMART_* и делimiters для вложенности.

N3. Воспроизводимость развёртывания. Полная система поднимается одной командой *docker compose up -d* из корня репозитория. Образ собирается по Dockerfile в два этапа (*builder + runtime*), запускается под пользователем без прав root, экспонирует порт 8000 и объявляет *healthcheck /api/stats*. *docker-compose.yml* смонтирует *SmartParking/frames/* как *drop-zone UE5, Models/* - как каталог весов, директории калибровок - в *read-only*-режиме, директорию *logs/* - на запись. База *barrier.db* сохраняется в именованном томе *barrier_db*, переживающем *docker compose down*.

N4. Модульность и тестируемость. Компоненты разделены на независимые модули: конфигурация (*neosmart.config*), трекинг (*neosmart.tracking*), детектор (*web_app.detector*), контроллер шлагбаума (*web_app.barrier_controller*), слой доступа к базе (*web_app.barrier_db*), распознавание номеров (*web_app.plate_scanner*). Покрытие автотестами - 55 *pytest*-тестов в шести файлах: *test_config.py* (YAML и *env*-переопределения), *test_sort.py* (детерминизм трекера), *test_barrier_state_machine.py* (граф переходов FSM и консенсус), *test_barrier_db.py* (CRUD + сессии), *test_wrong_parking.py* (геометрия порогов), *test_api_smoke.py* (FastAPI *TestClient*).

N5. Устойчивость. Временное отсутствие кадра на одной камере не останавливает обработку остальных пяти: функция *receive_frame* выполняет до десяти повторов *cv2.imread* с интервалом 50 мс, *stream_frames* после тридцати подряд пропусков отдаёт *placeholder*-изображение, не прерывая сессию. *BarrierController* при отсутствии готового *PlateRecognizer* инициализируется в *no-op*-режим и не блокирует запуск сервера.

N6. Наблюдаемость. Для мониторинга состояния используется связка *structured logging* (*neosmart.logging_setup*): JSON-lines в *logs/app.log* для автоматической сборки и человекочитаемый формат в *stdout*. Уровень логирования задаётся в секции *logging.level*, *endpoint /api/stats* служит лёгким *health-check*-зондом для Docker и внешних систем мониторинга.

N7. Изоляция данных. База `barrier.db` хранится внутри контейнера и не экспонируется на сетевой периметр; доступ к таблицам `allowed_plates`, `access_log` и `parking_sessions` возможен только через аутентифицированные REST-маршруты. Лог доступа выполняет функцию аудита: каждая попытка въезда - разрешённая или отклонённая - фиксируется с привязкой к номеру, `timestamp` и причине решения.

3.2. Общая архитектура и зонирование

Сформулированные требования определяют структуру системы на двух уровнях: горизонтальном - последовательность стадий обработки одного кадра от захвата в виртуальной сцене до отображения в браузере; и вертикальном - распределение шести камер по четырём функциональным зонам. Оба уровня закреплены в архитектуре и делают поведение системы предсказуемым и расширяемым: замена источника кадров (UE5 -> реальный видеопоток) не затрагивает уровень перцепции, а добавление новой камеры ограничивается записью в конфигурационном файле.

3.2.1. Многоуровневая схема потоков данных

Общая схема потоков данных системы разделена на четыре уровня. Первый уровень - источник кадров: виртуальная сцена Unreal Engine 5, описанная в главе 2, либо, в перспективе, реальные IP-камеры с поддержкой RTSP-трансляции. Второй уровень - транспорт: файловая `drop-zone SmartParking/frames/`, куда источник складывает последний кадр каждой камеры в виде файла `camera{N}.png`. Третий уровень - перцепция: FastAPI-процесс, загружающий кадры из `drop-zone`, прогоняющий их через `ParkingDetector` и `BarrierController` и формирующий агрегированную статистику. Четвёртый уровень - представление: WebSocket-канал `/ws/stream`, передающий JPEG-кодированные кадры и JSON-статистику в браузерный дашборд, плюс REST-интерфейс `/api/*` для CRUD-операций и `health-check`.

Разделение на уровни решает три практические задачи. Во-первых, транспорт через файловую систему (второй уровень) развязывает частоту генерации кадров источником и частоту их потребления перцептивным пайплайном: источник всегда пишет «последний кадр», потребитель всегда читает «текущий». Временная остановка любой из сторон не приводит к потере кадров - после возобновления взаимодействия процесс продолжается с актуального состояния файловой системы. Во-вторых, уровень перцепции (третий) инкапсулирует все нетривиальные вычисления в едином процессе FastAPI, запущенном через lifespan-хук: детектор YOLOv8s, SORT-трекеры платной и дорожной зон и два контроллера шлагбаума предзагружаются в памяти при старте сервера и переиспользуются на последующих запросах. В-третьих, уровень представления (четвёртый) не содержит бизнес-логики: агрегация статистики, дедупликация camera1/camera2 и renderings выполняются в ParkingMonitor на стороне клиента - сервер отдаёт сырые данные по камерам.

Ключевая точка координации - корутина `stream_frames()`, запускаемая отдельным `asyncio-task` на каждое подключение WebSocket-клиента и повторяющаяся с интервалом 0,1 секунды. В каждой итерации корутина читает последний кадр каждой камеры из `FrameStorage`, вызывает `ParkingDetector.process_frame()` с соответствующим `camera_id` и, для барьерных камер, передаёт результат в `BarrierController`. Результаты сериализуются в JPEG (`quality = 70`) и base64, упаковываются в единый JSON-пакет вместе с объединённой статистикой и отправляются клиенту одним кадром WebSocket. Такой подход обеспечивает детерминированную синхронизацию всех шести камер на клиенте и устраняет рассинхронизацию, характерную для независимых `per-camera` стримов.

Отдельная нитка взаимодействия проходит между перцептивным уровнем и источником кадров для управления шлагбаумом. `BP_BarrierPoller` на стороне UE5 опрашивает эндпоинт `GET /api/barrier/entry` с частотой 1 Гц;

FastAPI возвращает команду open, close или idle, вычисленную текущим состоянием конечного автомата. По завершении Timeline-анимации сцена отправляет POST /api/barrier/entry/ue5_ack с полем event в {open_complete, close_complete}; BarrierController переходит в следующее состояние FSM только после получения подтверждения. Такая схема устраняет рассинхронизацию визуальной анимации стрелы и внутренней логики допуска.

3.2.2. Распределение камер по функциональным зонам

Шесть камер распределены по четырём функциональным зонам, закреплённым в секции cameras файла config/default.yaml. **Бесплатная парковка** покрывается камерами camera1 и camera2 с ролью parking: обе наблюдают одну и ту же площадку под разными углами; полигоны разметки размещены на каждой из них независимо, при агрегации статистики результаты дедулицируются на клиенте. **Платная парковка** наблюдается единственной камерой camera3 с ролью parking и включённым флагом tracking: true - на этой камере работает SORT-трекер платной зоны, обеспечивающий детекцию подозрительных стоянок. **Дорожная зона** - camera4 с ролью road: запускает отдельный SORT-трекер с иными параметрами и логику подсчёта направления по двум полигонам пересечения. **Шлагбаумная зона** образуется двумя камерами: camera5 с ролью barrier_plate отвечает за считывание номера в зонах приближения и чтения, camera6 с ролью barrier_safety - за мониторинг зоны безопасности под стрелой шлагбаума.

Ролевое назначение реализовано через pydantic-модель CameraSettings (neosmart/config.py) с перечислением CameraRole ∈ {parking, road, barrier_plate, barrier_safety}. Модель AppSettings не хранит явных списков - они вычисляются свойствами parking_camera_ids, road_camera_ids, barrier_camera_ids, tracking_camera_ids и camera_ids из одного и того же конфигурационного словаря. Дополнительный валидатор _validate_cameras

запрещает установку `tracking=true` на камерах с ролью, отличной от `parking`, - это предотвращает некорректное включение трекера на камере бесплатной парковки или барьерной камере, для которых применяется собственная логика. Добавление седьмой камеры к системе не требует модификации кода: достаточно записи вида `camera7: {role: parking}` в `yaml`-файле, после чего запуск сервера автоматически регистрирует её в `derived`-коллекциях, загружает `pickle`-полигоны `CarParkingSpace/camera7_parkings.p` и подключает в пайплайн.

3.3. Модуль детекции и определения занятости мест

Модуль реализует требования **F1** и **F2**: покадровую детекцию автомобилей на всех шести камерах и определение статуса каждого размеченного парковочного места на камерах бесплатной и платной зон. Модуль воплощён в классе *ParkingDetector* (*SmartParking/web_app/detector.py*); общая точка входа - метод `process_frame(img, camera_id)`, вызываемый корутиной `stream_frames` на каждый кадр каждой камеры.

Разметка парковочных мест выполнена на подготовительном этапе через интерактивный инструмент *SmartParking/CarParkingSpace/mark_parking_spaces.py*. Инструмент открывает кадр выбранной камеры в окне `OpenCV` и принимает управление мышью: левый клик добавляет точку к текущему полигону, правый - снимает последнюю, четыре последовательных левых клика замыкают один четырёхугольник. Клавиша `Z` откатывает последний сохранённый полигон, клавиша `S` сохраняет текущее состояние в файл `{camera_id}_parkings.p` в той же директории. Формат файла - `pickle`-сериализованный список списков кортежей (x, y) в пиксельных координатах кадра 1280×720 . Такой формат прост для отладки (разбирается средствами стандартной библиотеки) и переживает любые изменения реализации *ParkingDetector*.

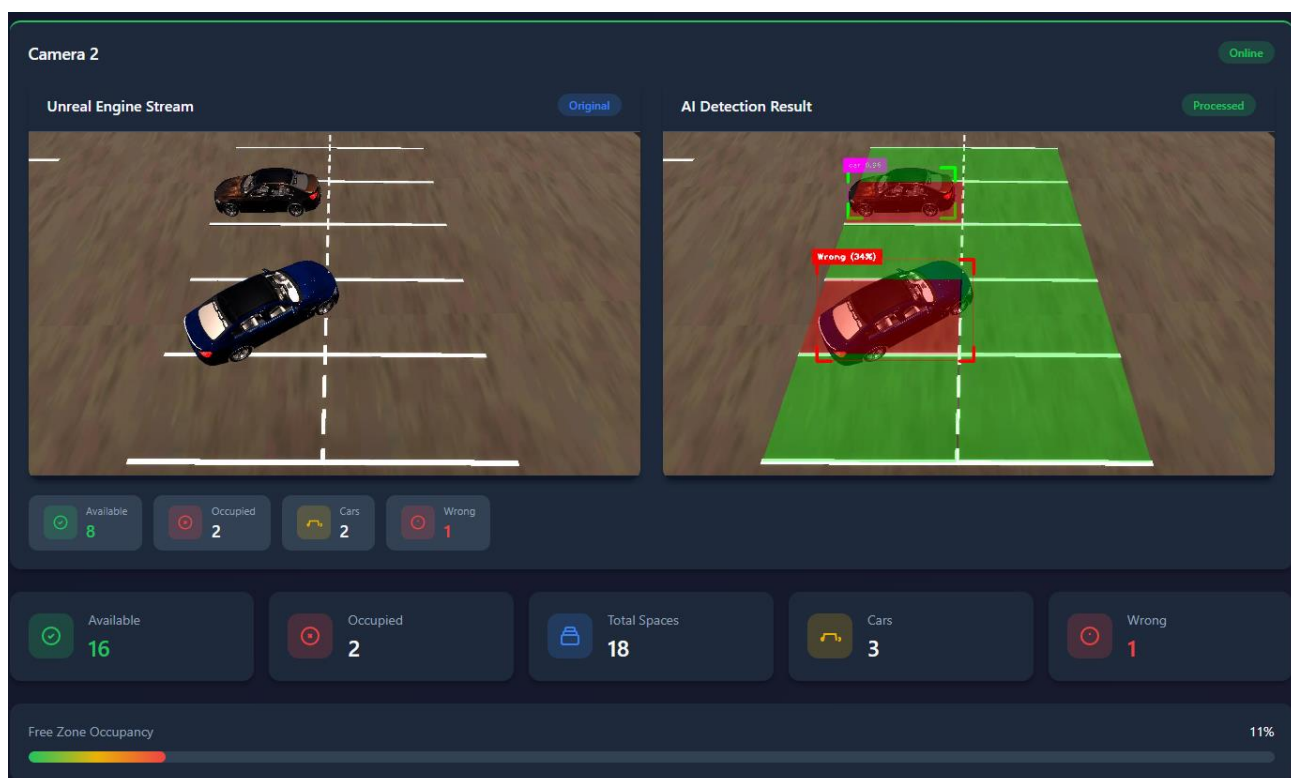
Подготовленные полигоны загружаются конструктором `ParkingDetector.__init__` в словарь `camera_polygons`, где ключ - `camera_id`, значение - список полигонов. Барьерные и дорожные камеры в этом словаре получают пустой список: для них применяется другая логика (см. 3.6, 3.7). Для каждой парковочной камеры фиксируется её `total_spaces` - число размеченных полигонов, - используемое при расчёте `available = total_spaces - occupied`.

Алгоритм оценки занятости реализован в методе `overlay_parking_spaces`. На каждом кадре для полигона P_i и каждого детектированного автомобиля с центром рамки c_j выполняется тест `cv2.pointPolygonTest(P_i, c_j, False)`. Положительный или нулевой результат - центр попал в полигон, автомобиль считается занимающим место P_i ; полигон заливается красным, счётчик `occupied_count` увеличивается. В отсутствие такого автомобиля полигон заливается зелёным. Заливка полупрозрачная (`alpha = 0,35`, `cv2.addWeighted`), что сохраняет видимость содержимого кадра под полигоном. Структура цикла оценки занятости воспроизведена в листинге 3.1.

```
for i, polygon in enumerate(polygons):
    poly_np = np.array(polygon, np.int32).reshape((-1, 1, 2))
    car_in_spot = None
    for obj in object_list:
        if cv2.pointPolygonTest(poly_np, obj['center'], False) >= 0:
            car_in_spot = obj
            break
    if car_in_spot is None:
        cv2.fillPoly(overlay, [poly_np], (0, 255, 0)) # free
    else:
        cv2.fillPoly(overlay, [poly_np], (0, 0, 255)) # occupied
        occupied_count += 1
        # ... проверка wrong parking см. 3.4
cv2.addWeighted(overlay, 0.35, img, 0.65, 0, img)
```

Формально статус занятости и число занятых мест на камере c задаются выражениями $occupied(c) = |\{ i \in [0, |P(c)|) : \exists j. c_j \in P_i \}|$ и $available(c) = |P(c)| - occupied(c)$, где $P(c)$ — множество полигонов камеры c , c_j - центр рамки j -го автомобиля на кадре. Вычислительная сложность - $O(|P(c)| \cdot |D(c)|)$, где $|D(c)|$ - число детекций YOLO на кадре; для типичной сцены с $|P| \leq 20$ и $|D| \leq 10$ полный цикл оценки занимает порядка 0,3 мс и не составляет доминирующей доли бюджета кадра (ключевая стоимость - инференс YOLO).

Дедупликация бесплатной зоны учтена на уровне агрегации статистики. Эндпоинт GET /api/stats в main.py суммирует total_spaces и occupied по всем парковочным камерам, но счётчик cars_detected аккумулирует только значение, полученное на camera1 - camera2 наблюдает тот же физический автомобиль под другим ракурсом, и простое суммирование привело бы к удвоению. Аналогичная дедупликация выполнена в ParkingMonitor на стороне браузера при расчёте сводных карточек для overview-страницы дашборда.



[Рисунок 3.1 - Скриншот дашборда с цветовой индикацией свободных и занятых мест на бесплатной зоне]

3.4. Модуль обнаружения неправильной парковки

Модуль реализует требование F3 и интегрирован в метод overlay_parking_spaces рядом с логикой оценки занятости: обнаружение нарушения выполняется только для полигонов, у которых зафиксировано занятое состояние - иначе понятие «неправильной парковки» не имеет смысла. Постановка задачи: определить, когда рамка автомобиля значительно выходит за границы размеченного полигона, и отличить такой случай от корректной

парковки, при которой небольшое выступление рамки объясняется погрешностью разметки либо проекционным искажением.

Метрика нарушения - процент площади рамки, находящейся вне полигона парковочного места, - определяется формулой $pct(P, bbox) = (A_{bbox} - A_{\cap}(P, bbox)) / A_{bbox} \cdot 100 \%$, где A_{bbox} - площадь рамки в пикселях, равная $w \cdot h$, а $A_{\cap}(P, bbox)$ - площадь пересечения полигона P и рамки (рисунок 3.2). При отсутствии пересечения $pct = 100 \%$, при полном вхождении рамки в полигон $pct = 0 \%$.

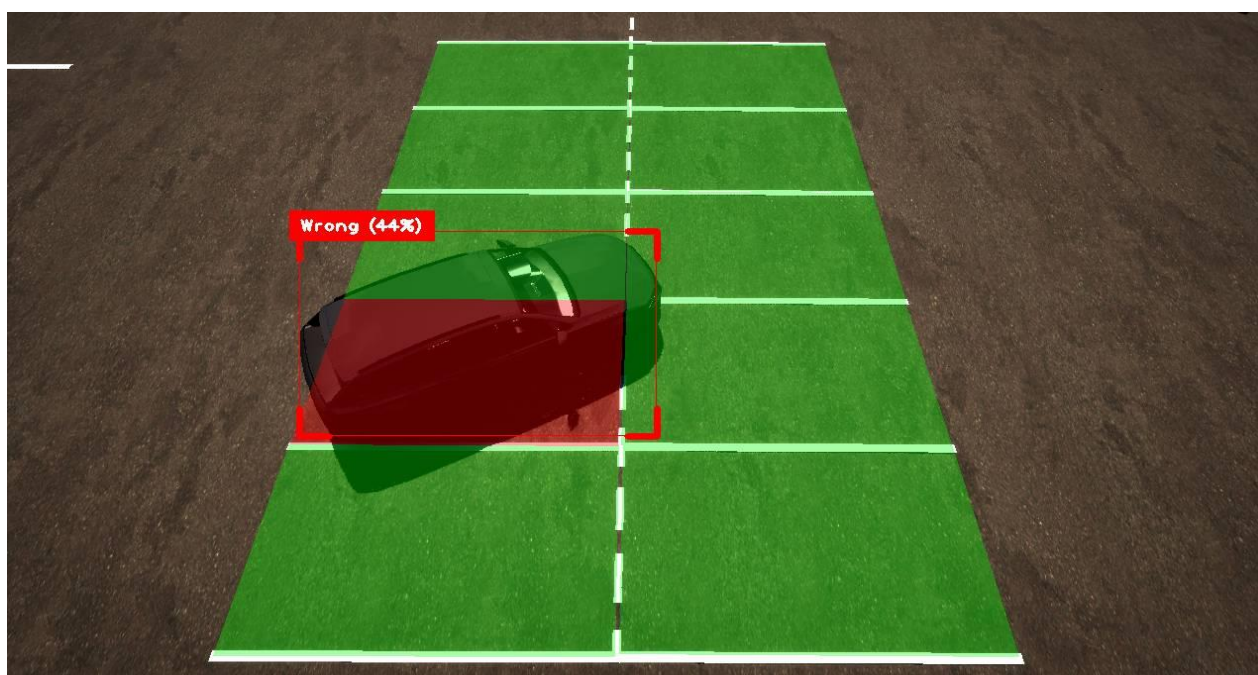


Рисунок 3.2 - Визуализация метрики *outside_percentage* в UE5-сцене: красная заливка *bbox* соответствует автомобилю, припаркованному поперёк разметки; метка «Wrong (44%)» выводится при превышении адаптивного порога

Реализация расчёта, представленная в методе `_compute_outside_percentage`, избегает аналитического клиппинга произвольного полигона и прямоугольника - вычисление выполняется на пиксельных масках. Создаётся двумерная булева маска `poly_mask` функцией `cv2.fillPoly` и прямоугольная маска рамки `car_mask` функцией `cv2.rectangle`; площадь пересечения равна числу ненулевых пикселей побитового `cv2.bitwise_and`. Растровый подход вычислительно устойчив к любой форме полигона (выпуклый, невыпуклый, самопересекающийся), не требует

отдельной библиотеки для геометрических операций и выполняется за 0,7–0,9 мс на кадре 1280×720 в расчёте на один полигон - это один-два процента бюджета кадра.

Критерий нарушения - превышение порога, зависящего от камеры и индекса полигона: $\text{pct}(P_i, \text{bbox}_j) > T(c, i)$. Порог T не является константой: широкоугольные камеры на высоте 4–6 м дают перспективные и fisheye-искажения, при которых проекция корректно припаркованного автомобиля у края кадра визуально «раздувается» и без компенсации даёт ложное срабатывание. Предложенная схема резолуции порогов реализована классом `WrongParkingSettings` (`neosmart/config.py`, метод `resolve_threshold`) и содержит четыре уровня приоритета, проверяемых в порядке убывания.

1. `Per-index override` - порог, привязанный к конкретному индексу полигона на конкретной камере. В текущей конфигурации заданы `camera1 : [0, 1] → 52 %`, `camera3 : [0, 1] → 46 %` - это крайние полигоны с наиболее сильным проекционным искажением. Переопределение заведомо побеждает любое правило нижних уровней.

2. `Edge-polygon rule` - правило, выделяющее группу полигонов по близости к краю кадра. Конфигурация задаёт `camera1 : {first_n = 2, last_n = 2}` - первые и последние два полигона получают повышенный порог `outside_threshold_edge = 45 %`; `camera2 : {last_n = 2}` - последние два полигона. Edge-правило применяется, только если индекс не перекрыт `per-index override`.

3. `Per-camera default` - порог, единый для всех полигонов камеры, но сдвинутый относительно глобального значения. В конфигурации заданы `camera2 → 31 %` и `camera3 → 36 %`: эти камеры наблюдают парковку под большим углом к горизонту, чем `camera1`, и базовый порог 25 % даёт устойчивые ложные срабатывания.

4. Глобальный `default` - `outside_threshold_default = 25 %`. Используется для всех оставшихся полигонов, не охваченных предыдущими тремя уровнями.

Все четыре уровня сконфигурированы в секции `wrong_parking` файла `config/default.yaml` и валидированы `pydantic`-моделью: проценты ограничены диапазоном $[0, 100]$, индексы в `per-index override` - неотрицательные целые, структура `edge`-правил типизирована через `EdgeRule`. Подобная иерархия даёт три преимущества. Во-первых, корректировка ложных срабатываний на конкретном полигоне не затрагивает остальные места: достаточно добавить одну запись в `index_overrides`. Во-вторых, пороги хранятся в репозитории рядом с кодом и версионируются через `git` - в отличие от файлов калибровки камер, которые обычно являются бинарными `pickles` вне системы контроля версий. В-третьих, переход на новую камеру не требует полной `intrinsic`-калибровки объектива: для нового объекта достаточно эмпирически подобрать два-три числа `per-camera` и, при необходимости, точно поднять пороги для крайних полигонов.

Сравнение с альтернативными методами, рассмотренными в 1.4.4, показывает, что выбранный растровый подход остаётся оптимальным для задачи настоящей работы. Гомографический подход требует определения восьми параметров проективного преобразования и коррекции радиальной дисторсии - для каждой камеры и каждой её перестановки отдельно; такой вариант не соответствует требованию N2 (горизонтальное масштабирование через конфигурацию). Сравнение `aspect-ratio` рамки автомобиля с эталоном даёт неустойчивое поведение при частичных окклюзиях и вариативной ориентации автомобиля (параллельная, перпендикулярная, угловая парковка). Бинаризация области полигона теряет качество под уличными условиями с резкими перепадами освещения - этот эффект продемонстрирован на ночных сценах полигона в 2.1.4.

Визуализация нарушения осуществляется в обработанном кадре: рамка нарушителя перерисовывается красным поверх стандартной зелёной (функция `cvzone.cornerRect` с `colorR = (0, 0, 255)`), над или под рамкой помещается подпись вида `Wrong`, указывающая точное значение метрики. Количество

нарушителей на кадре возвращается в статистике ключом `wrong_count` и транслируется в дашборд - в карточке зоны отображается счётчик, при выходе отдельной рамки за порог включается мигание соответствующего индикатора на стороне клиента.

Корректность реализации проверена автоматическими тестами в `tests/test_wrong_parking.py`: фиксируются граничные случаи (рамка полностью внутри полигона даёт `pct = 0`, полностью вне - `pct = 100`), совпадение растрового расчёта с аналитическим для прямоугольного полигона в пределах одного процента, а также независимая проверка четырёх уровней резолуции порогов - по одному тесту на уровень. Такой набор тестов защищает от регрессий при последующих изменениях конфигурации и гарантирует сохранение контрактов модуля при подключении новых камер.

3.5. Модуль трекинга и детекции подозрительной стоянки

Модуль реализует требования **F4** и **F5**: присвоение каждому автомобилю в платной зоне устойчивого идентификатора между кадрами и фиксацию превышения допустимой длительности стоянки. Трекинг включён только на одной камере - `camera3`, - что закреплено в конфигурации: роль `parking` с флагом `tracking: true` присваивается в `config/default.yaml`, а `derived-view AppSettings.tracking_camera_ids` возвращает именно эту камеру. На бесплатной зоне трекинг не нужен - там достаточно покадровой оценки занятости, а стоимость запуска трекера на двух камерах бесплатной и шестой барьерной не окупается.

Выбор алгоритма. Как показано в разделе 1.4.2, для стационарных камер с видом сверху и редкими окклюзиями SORT оказывается предпочтительнее DeepSORT или ByteTrack: его вычислительные затраты ограничиваются одним шагом фильтра Калмана на трек и одним назначением венгерским алгоритмом на кадр, а `re-identification` по визуальному эмбедингу, ради которого обычно добавляют DeepSORT, здесь избыточен - ракурс каждой

камеры фиксирован, внешний вид автомобилей между соседними кадрами почти не меняется. Канонический SORT вынесен в общий пакет `neosmart/tracking/sort.py` и импортируется в `ParkingDetector` оттуда; дублирующая реализация в коде приложения отсутствует.

Параметры трека. Конструктор SORT принимает три значения, взятых из `TrackingSettings` (`neosmart/config.py`): `sort_max_age` = 20 кадров (приблизительно две секунды на частоте 10 Гц) - верхняя граница времени удержания трека без подтверждающей детекции; `sort_min_hits` = 5 кадров - минимум совпадающих детекций до того, как трек считается подтверждённым и его идентификатор выводится на кадр; `iou_threshold` = 0.3 - порог IoU для ассоциации детекции с существующим треком. Эти значения подобраны под частоту кадров и скорости на парковочной камере: более короткое `sort_max_age` приводит к дроблению трека при кратковременной окклюзии, большее `sort_min_hits` задерживает появление идентификатора настолько, что оператор не успевает связать рамку с машиной до её остановки.

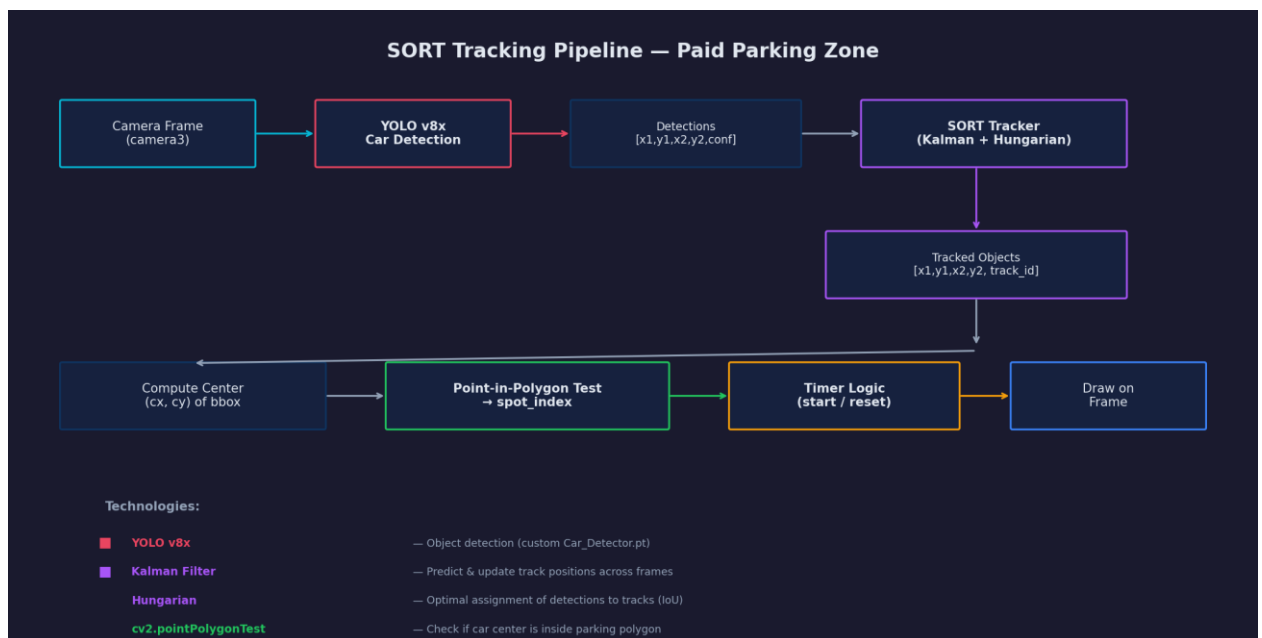


Рисунок 3.3 - Пайплайн трекинга SORT на платной зоне: детекции YOLOv8s преобразуются в формат `[x1, y1, x2, y2, conf]`, пропускаются через фильтр Калмана и венгерский алгоритм, затем центры подтверждённых треков проверяются на попадание в полигоны парковочных мест

Идентификация автомобилей реализована в два уровня. Внутренний `sort_id`, выдаваемый трекером, монотонно растёт и никогда не сбрасывается,

что удобно для внутренних структур, но неудобно для оператора: номер 173 спустя час наблюдения ни о чём не говорит. Модуль поддерживает вторую таблицу `camera_display_ids`, в которой `sort_id` отображается в последовательные видимые номера 1, 2, 3, ... с момента старта сервера. На дашборде машина отображается именно с `display_id` - это делает журнал событий читаемым и соответствует визуальному ожиданию зрителя.

Детекция подозрительной стоянки основана на таймере длительности пребывания автомобиля внутри размеченного полигона. Для каждого активного трека метод `_update_tracking` вычисляет центр рамки и выполняет `test cv2.pointPolygonTest` по списку полигонов `camera3`. Если центр попадает в полигон P_i и трек ещё не был зафиксирован в этом месте, словарь `track_times[display_id]` получает пару $(start_time, i)$. При каждом последующем кадре, пока центр остаётся в том же полигоне, таймер увеличивается; если автомобиль переместился в другой полигон, пара перезаписывается с новым `start_time` - счёт времени начинается с нуля. Выход центра за пределы всех полигонов удаляет запись целиком: короткие манёвренные пересечения не накапливают сомнений.

Формально статус трека n на момент времени t определяется как $suspicious(n, t) = [t - start_time(n) \geq T_susp] \cdot [spot(n, t) \neq \emptyset]$, где T_susp - пороговое время подозрительной стоянки, заданное параметром `tracking.suspicious_time_threshold` (по умолчанию 30 секунд). Значение 30 секунд выбрано как компромисс между чувствительностью и устойчивостью к кратковременному паркингу.

Визуализация различает три состояния. Автомобиль, обнаруженный, но не попавший в полигон, получает голубую подпись `ID:N` и никакой рамки поверх базовой YOLO-обводки. Как только центр входит в полигон, рамка перекрашивается синим, а под ней появляется подпись `ID:N Xs` - где X - накопленное время стоянки. После превышения T_susp рамка и подпись становятся красными, а справа от рамки появляется вертикальная плашка

SUSPICIOUS с перелимитом +M:SS; её положение проверяется на выход за границы кадра и при необходимости смещается влево от рамки. Именно красная рамка и плашка - то, на что в первую очередь реагирует оператор дашборда.

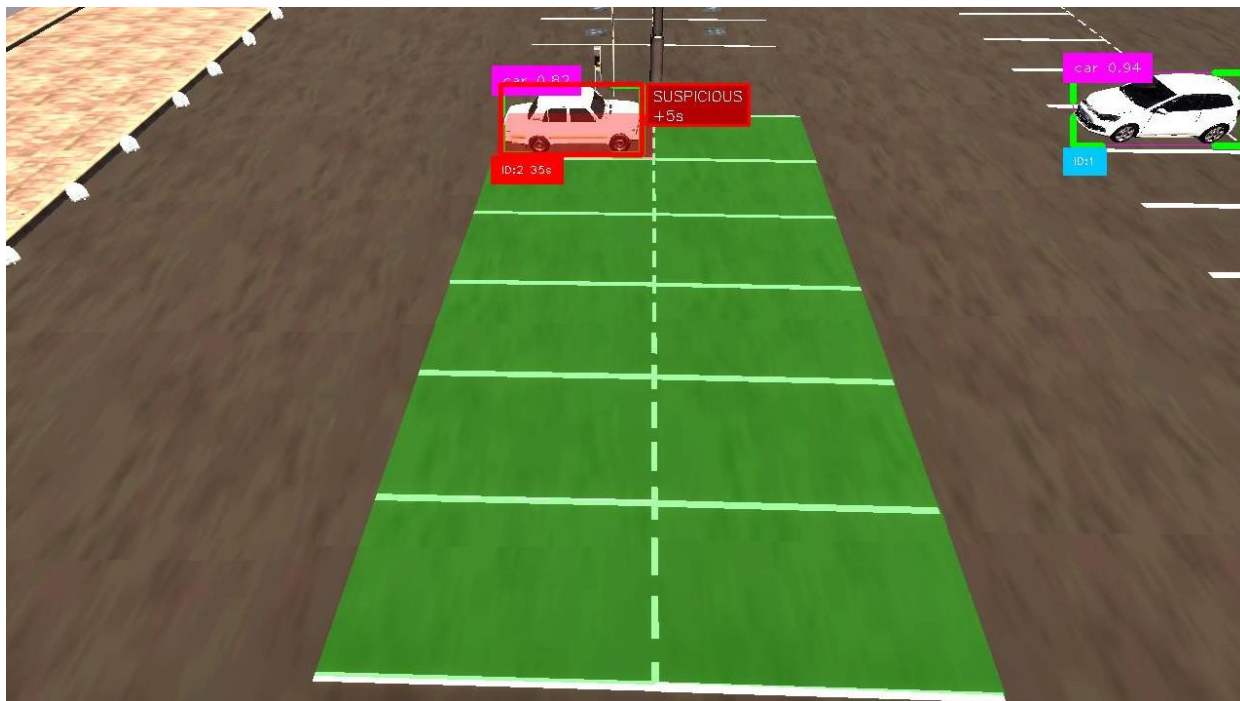


Рисунок 3.4 - Пример работы модуля на сцене UE5: справа - автомобиль в состоянии Tracked с голубой подписью идентификатора; справа - автомобиль, превысивший порог $T_{susp} = 30$ с, с красной рамкой и плашкой SUSPICIOUS, вынесенной за пределы рамки

Уборка устаревших структур выполняется в каждом вызове `_update_tracking`: из `track_times` удаляются записи, которых нет среди активных `display_id` текущего кадра, а из `display_ids` - маппинги для `sort_id`, отсутствующих в возврате трекера. Такая двусторонняя очистка предотвращает рост словарей при долгой работе сервера и гарантирует, что трек, возобновлённый SORT после долгой окклюзии, попадёт в систему как новый - с обнулённым таймером.

Корректность модуля зафиксирована в `tests/test_sort.py`. Покрыты детерминизм трекера на фиксированных входных данных, корректность матчинга при IoU-пересечениях разной степени, сохранение идентификатора при двух-трёхкадровых пропусках детекции, уничтожение трека после `sort_max_age` без подтверждений. Отдельные кейсы проверяют таймерную

логику: запуск таймера при входе в полигон, обнуление при смене места, переход в *suspicious* по превышению порога.

3.6. Модуль подсчёта транспортного потока

Модуль реализует требование **F6**: отдельный подсчёт автомобилей, движущихся ко въезду на территорию и от неё, по камере *camera4*, установленной на прилегающей дороге. Роль *road* закрепляется в *config/default.yaml* и автоматически формирует *derived-view* *AppSettings.road_camera_ids*; метод *process_frame* в *ParkingDetector* для *road*-камер вызывает *_update_traffic_tracking* вместо *overlay_parking_spaces*.

Калибровка. Две полигональные области - *incoming* и *outgoing* - размечаются оператором в инструменте *SmartParking/TrafficCounting/mark_crossing_regions.py*. Инструмент открывает кадр *camera4*, ожидает последовательность кликов по вершинам первого, затем второго полигона и сохраняет результат в *camera4_crossing.p*. Конструктор *ParkingDetector* подгружает этот файл в *self.crossing_regions["camera4"] = {"incoming": [...], "outgoing": [...]}*.

Параметры трекера для дорожной сцены отличаются от параметров платной парковки - автомобили движутся быстрее, частично перекрываются столбами и чаще находятся близко к границе кадра. Поэтому в *TrackingSettings* заданы отдельные значения: *traffic_sort_max_age = 50* кадров (пять секунд) - чтобы не потерять трек при кратковременной окклюзии на обочине; *traffic_sort_min_hits = 0* - трек подтверждается первой же детекцией, иначе быстрый автомобиль успеет пересечь линию до подтверждения; *traffic_iou_threshold = 0.15* - менее строгий порог ассоциации компенсирует резкие изменения рамки из-за перспективы.

Для каждого активного трека, полученного от SORT, вычисляется центр рамки и проверяется на попадание в каждый из двух полигонов направлений. Если центр впервые оказывается внутри полигона *incoming* или *outgoing*

(условие `display_id not in crossed[direction]`), счётчик соответствующего направления увеличивается на единицу, а идентификатор заносится в множество пересёкших линию. Это гарантирует, что одна и та же машина не учитывается дважды - даже если она остановилась поверх линии и задержалась там на десятки кадров. Для визуальной обратной связи полигон кратковременно, в течение одной секунды, заливается красным цветом в момент регистрации нового пересечения.

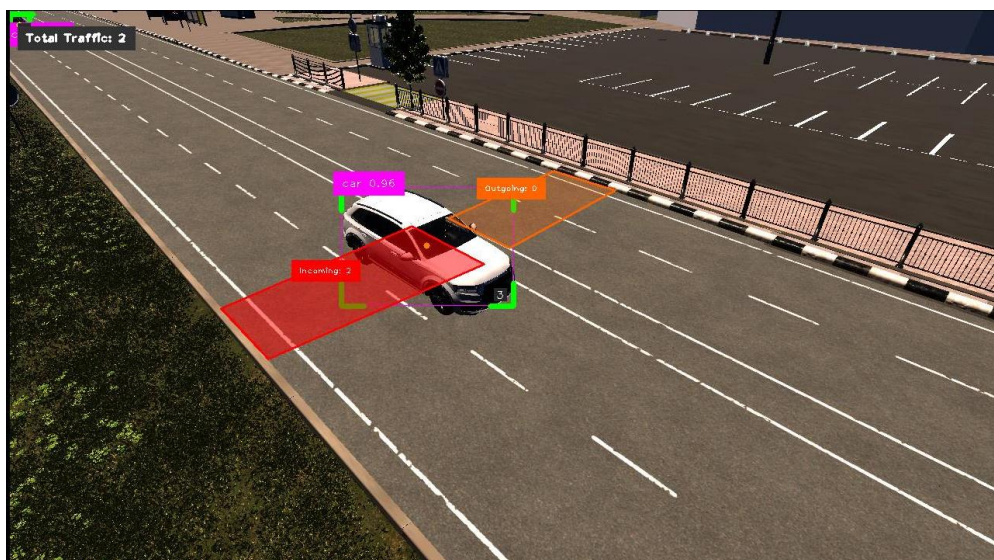


Рисунок 3.5 - Работа модуля подсчёта транспортного потока: красный полигон *incoming* у въезда на территорию (из-за пересечения его автомобилем), оранжевый полигон *outgoing* для обратного направления, счётчики направлений в верхнем левом углу кадра

Итоговые значения `incoming_count` и `outgoing_count` возвращаются в `stats`-словаре `process_frame` и передаются на дашборд как отдельные метрики зоны `road`. Тестовая сьюта не содержит выделенного файла для трафика - общая валидность SORT-ассоциатора покрыта в `test_sort.py`, а корректность учёта пересечений вручную проверялась на записях сценария с предопределённым числом проездов.

3.7. Модуль управления входным шлагбаумом

Модуль реализует требование **F7** и является самой сложной подсистемой работы. Он объединяет детекцию автомобилей, распознавание регистрационных знаков, проверку доступа по базе данных и управляющий сигнал, возвращаемый в Unreal Engine 5, в одну согласованную процедуру с

явно определёнными состояниями. Ниже последовательно рассмотрены семь компонентов: двухкамерная схема наблюдения, разметка зон, конечный автомат, консенсусная логика OCR, окно ожидания зоны безопасности, протокол взаимодействия с UE5 и структура базы данных доступа.

3.7.1. Двухкамерная схема наблюдения

Барьер наблюдается двумя камерами, разведёнными по функциональной нагрузке. Camera5 (роль `barrier_plate`) ориентирована на полосу подъезда: её задача - увидеть машину до шлагбаума и считать её государственный регистрационный знак. Camera6 (роль `barrier_safety`) ориентирована на пространство под стрелой и за ней: её задача - подтвердить, что зона безопасности свободна, когда пора закрывать шлагбаум. Конфигурация фиксирует эти роли в `barrier.entry_plate_camera = camera5` и `barrier.entry_safety_camera = camera6`; `AppSettings.barrier_camera_ids` возвращает кортеж из двух идентификаторов.

Разделение на две камеры - осознанное решение. Одна камера, расположенная над шлагбаумом, теряет машину сразу после въезда (уходит под козырёк объектива) и не может контролировать зону безопасности во время закрытия; одна камера, расположенная вне зоны шлагбаума, не способна считать номер до открытия. Два ракурса с разных сторон закрывают обе фазы цикла без компромиссов.



Рисунок 3.6 – Кадр с camera5 двухкамерной схемы наблюдения шлагбаума: зоны APPROACH и READING, в текущем состоянии READING_PLATE зона READING подсвечена жёлтым и демонстрирует успешное распознавание номерного знака E232BA152; в верхней части - индикатор состояния автомата

3.7.2. Разметка зон приближения, считывания и безопасности

Три зоны размечены оператором на этапе калибровки через инструмент SmartParking/BarrierSystem/mark_barrier_zones.py. Инструмент запускается для каждой барьерной камеры, последовательно предлагая разметить три полигональные области, и сохраняет их в файлы camera5_barrier.p (зоны APPROACH и READING) и camera6_barrier.p (зона SAFETY). BarrierController на старте загружает эти полигоны, преобразует их в массивы numpy формата $(-1, 1, 2)$ и использует в cv2.pointPolygonTest - стандартной проверке «точка-в-многоугольнике», возвращающей знаковое расстояние до границы.

Зона APPROACH задаёт узкий участок подъезда - в ней автомобиль впервые становится видимым для контроллера, но ещё не имеет оснований инициировать сканирование. Зона READING расширена и расположена непосредственно перед стрелой: именно там положение номера относительно камеры пригодно для OCR, и именно попадание центра рамки в READING переводит автомат в состояние сканирования. Зона SAFETY на camera6 описывает пространство прямо под стрелой и небольшую полосу за ним - это тот участок, на котором закрывать шлагбаум физически опасно.

3.7.3. Конечный автомат барьерного контроллера

Поведение контроллера формализовано как конечный автомат с девятью состояниями, хранящимися как строковые константы в модуле `barrier_controller.py`:

- **IDLE** - автомобилей в зонах нет, шлагбаум закрыт, команда UE5 равна `idle`; при появлении машины в **READING** контроллер пропускает **CAR_APPROACHING** и сразу переходит к чтению, закрывая класс случаев «въезд произошёл во время предыдущего цикла закрытия».
- **CAR_APPROACHING** - автомобиль замечен в **APPROACH**, но ещё не достиг зоны чтения; выход из состояния возможен в **READING_PLATE** (центр вошёл в **READING**) или в **IDLE** (машина покинула **APPROACH** и провела вне зоны более 3 секунд).
- **READING_PLATE** - ключевое состояние, в котором на каждом кадре запускается `PlateRecognizer.detect_plate` по рамке автомобиля, а результат накапливается в `ocr_buffer`; выход - при консенсусе или при уходе машины из **READING**.
- **ACCESS_GRANTED** / **ACCESS_DENIED** - транзитные состояния, в которых фиксируется решение и запись в журнал; **ACCESS_DENIED** удерживается `ACCESS_DENIED_TIMEOUT = 5` секунд, чтобы оператор успел увидеть результат; **ACCESS_GRANTED** немедленно выставляет `barrier_command = open` и переходит к следующему состоянию.
- **BARRIER_OPENING** - команда `open` удерживается до подтверждения от UE5, чтобы потеря одного сообщения на `polling`-интервале не приводила к несогласованному поведению; таймаут `BARRIER_OPENING_TIMEOUT = 30` секунд предохраняет автомат от зависания при проблемах с сетью.

- **CAR_PASSING** - автомобиль проезжает под шлагбаумом; зона безопасности проверяется через данные с camera6 (см. 3.7.5), и именно это состояние реализует безопасное ожидание.
- **BARRIER_CLOSING** - симметрично **BARRIER_OPENING**: команда close удерживается до close_complete-подтверждения или до таймаута в 30 секунд, после чего автомат возвращается в IDLE.

Переходы между состояниями инкапсулированы в методе `_set_state`, который записывает время входа в состояние (`state_enter_time`) и сбрасывает специфические буферы: `ocr_buffer` при входе в **READING_PLATE**, `safety_clear_time` при входе в **CAR_PASSING**, `barrier_command` и `last_access_result` при входе в **IDLE**. Сохранение момента входа позволяет во всех местах автомата сравнивать `elapsed_in_state = now - state_enter_time` с явно заданными порогами и избегать неявных таймеров, рассеянных по коду.

3.7.4. Консенсусный алгоритм распознавания номера

Обычная схема «одно распознавание - одно решение» непригодна по двум причинам. Во-первых, OCR на одиночном кадре мерцает: ParseQ в 5–15 % случаев выдаёт различающиеся на одну–две буквы варианты для подряд идущих кадров одной машины. Во-вторых, цена ошибки асимметрична: ложное открытие потенциально пропускает неавторизованную машину, а ложное закрытие только вежливо заставляет водителя сдать назад. Поэтому модуль применяет асимметричную консенсусную логику.

Накопление. Каждый кадр, пока центр рамки автомобиля находится в зоне **READING**, метод `_accumulate_ocr` вызывает `PlateRecognizer.detect_plate` для текущего кадра и кропа по рамке машины. Результат - словарь с полями `plate_text`, `confidence` и `cropped`-изображением самой номерной рамки - добавляется в `ocr_buffer`. Порог доверия `PLATE_CONFIDENCE_THRESHOLD = 0.3` отфильтровывает заведомо шумные чтения ещё внутри `PlateRecognizer`.

Асимметричное решение. При каждом новом чтении выполняется моментальная проверка: если распознанный `plate_text` присутствует в таблице `allowed_plates`, доступ предоставляется немедленно, без ожидания консенсуса (метод `_grant_access`). Обоснование: положительное совпадение с `whitelist` требует точного совпадения строк, что делает случайный `false grant` редким событием; к тому же нет смысла продолжать накопление и задерживать открытие шлагбаума для легитимной машины. Отказ, напротив, требует консенсуса: метод `_has_plate_consensus` возвращает `True`, если среди накопленных чтений найден вариант, встретившийся не менее `PLATE_MIN_AGREEING_READS = 3` раз. Только после достижения этого условия или при выходе машины из зоны `READING` контроллер вызывает `_decide_access`, который по большинству голосов выбирает результирующую строку и соответствующее чтение с максимальной уверенностью.

Выход из состояния. Если автомобиль покинул `READING` до накопления трёх совпадающих чтений и `ocr_buffer` непуст, решение всё равно принимается по имеющимся данным - иначе пустой буфер ведёт к возврату в `IDLE`, и цикл начинается заново при следующем въезде. Такой «мягкий» выход соответствует реальному поведению водителей, которые иногда сдают назад, заметив запрет.

Корректность консенсусной логики покрыта файлом `tests/test_barrier_state_machine.py`: тесты фиксируют немедленный `grant` при попадании в `whitelist` первым же чтением, удержание в `READING_PLATE` при одиночных шумных чтениях, переход в `ACCESS_DENIED` при консенсусе на неизвестном `plate_text`, выход в `IDLE` при пустом буфере.

3.7.5. Окно ожидания зоны безопасности

Сразу после того, как `UE5` подтверждает `open_complete`, контроллер переходит в `CAR_PASSING` и должен решить, когда закрывать шлагбаум. Наивная логика «закрывай, когда зона `SAFETY` пуста» приводит к гонке: автомобиль, только что миновавший плейт-камеру, ещё не успел доехать до

safety-зоны, и контроллер закрывает шлагбаум ему в корпус. Модуль предотвращает это тремя вложенными таймерами.

- **Grace period:** SAFETY_GRACE_PERIOD = 3.0 секунды с момента входа в CAR_PASSING - интервал, в течение которого состояние safety не проверяется вовсе. Значение подобрано под расстояние между плейт-камерой и стрелой в текущей сцене UE5 (≈ 5 м при скорости 10 км/ч).
- **Clear delay:** SAFETY_ZONE_CLEAR_DELAY = 1.0 секунда - как только зона SAFETY опустела, контроллер ждёт ещё одну секунду без входа автомобилей, прежде чем выдать команду close. Задержка фильтрует кратковременные пропуски детекции.
- **Open timeout:** BARRIER_OPEN_TIMEOUT = 30 секунд - максимальное время пребывания в CAR_PASSING без обнаружения автомобиля в зоне безопасности; по истечении контроллер принудительно переходит к закрытию, защищая систему от ситуации, когда водитель развернулся и уехал до въезда.

Источник данных для зоны SAFETY - отдельный: BarrierController принимает детекции с camera6 через метод update_safety, который вызывается ParkingDetector.process_frame для барьерной safety-камеры. Это развязывает цикл обработки camera5 (plate_recognition + advance_state) и camera6 (safety_check) - обе камеры обрабатываются параллельно в потоке stream_frames, но их результаты стекаются в один автомат через унифицированный интерфейс.

3.7.6. Интеграция с Unreal Engine 5

Стрела шлагбаума находится внутри виртуальной сцены, следовательно, сервер не может двигать её напрямую - ему нужно передать UE5 команду и дождаться, когда анимация завершится. Протокол реализован двумя маршрутами REST API.

- GET /api/barrier/{barrier_id} - UE5 опрашивает этот эндпоинт раз в секунду; ответ - JSON с полями state, command ∈ {idle, open, close}, barrier_position ∈ {closed, opening, open, closing}, last_plate, access. При command = open UE5 запускает анимацию поднятия, при command = close - опускания.
- POST /api/barrier/{barrier_id}/ue5_ack с телом {event: open_complete} или {event: close_complete} - подтверждение окончания анимации. Метод ack_from_ue5 переводит автомат в следующее состояние (CAR_PASSING после open_complete, IDLE после close_complete) и снимает липкую команду, возвращая barrier_command в idle.

Липкость команды, упомянутая в 3.7.3, - ключевая особенность протокола. Если UE5 по какой-то причине пропускает ответ сервера (например, опрос пришёлся ровно на кадр рендеринга, когда ответ задержался), команда останется в barrier_command на следующие секунды и будет повторно доставлена. Это делает интеграцию устойчивой к джиттеру без дополнительного механизма очередей.

На случай полного отсутствия подтверждений (например, остановка UE5) контроллер не виснет: таймауты BARRIER_OPENING_TIMEOUT и BARRIER_CLOSING_TIMEOUT по 30 секунд принудительно переводят автомат дальше, а в журнал пишется предупреждение с указанием того, что рассогласование состояний произошло по инициативе сервера.

3.7.7. Схема базы данных доступа

Хранилище барьерного модуля - SQLite-файл data/barrier.db, инкапсулированный в классе BarrierDatabase (barrier_db.py). База создаётся автоматически при первом старте, работает в режиме WAL (Write-Ahead Logging) для уменьшения конкуренции на запись и содержит три таблицы.

- *allowed_plates* - whitelist государственных регистрационных знаков; поля id, plate_number (UNIQUE, COLLATE NOCASE),

owner_name, vehicle_description, created_at, is_active. Сравнение регистронезависимо, что упрощает администрирование.

- *access_log* - журнал всех попыток въезда; поля id, timestamp, barrier_id, plate_number, confidence, access_result ∈ {granted, denied, manual_override}, frame_snapshot_path. Индексируется по timestamp для быстрой выборки последних записей на дашборде.
- *parking_sessions* - сессии пребывания автомобиля на территории; поля id, plate_number, entry_time, exit_time, duration_minutes, status ∈ {active, completed}. Индексируется парой (plate_number, status) - этого достаточно для эффективного поиска открытой сессии на завершение при повторном появлении номера.

Начальное заполнение whitelist реализовано через файл data/seed_plates.json: при пустой таблице allowed_plates BarrierDatabase._seed_from_file выполняет INSERT OR IGNORE для каждой записи с полями plate_number, owner_name, vehicle_description. Это удобно для демонстрационного стенда - набор знакомых номеров доступен сразу после первого запуска.

При старте сервера метод reset_runtime_state очищает access_log и parking_sessions, оставляя whitelist нетронутым. Такое поведение связано с назначением системы: журнал событий и активные сессии - это данные текущего запуска, и их накопление между запусками демонстрационного стенда искажает статистику на дашборде. В продуктивной установке этот вызов легко отключить через флаг в конфигурации.

CRUD-операции над whitelist вынесены в методы add_plate, remove_plate, get_allowed_plates и прокинуты наружу через REST-маршруты /api/barrier/plates (см. 3.8.2). Корректность CRUD и целостность индексов зафиксированы в файле tests/test_barrier_db.py: добавление, удаление, повторный импорт из seed, работу на регистронезависимое сравнение, запись события и закрытие активной сессии.

3.8. Серверная часть

3.8.1. Стек технологий и жизненный цикл приложения

Сервер реализован на FastAPI 0.128 под управлением Uvicorn; единственная HTML-страница дашборда собирается шаблонизатором Jinja2, статические ресурсы раздаются через ImmutableStaticFiles с cache-busting. Выбор FastAPI обусловлен встроенной валидацией pydantic-моделей, поддержкой WebSocket в одной кодовой базе с REST и наличием lifespan-хука. В lifespan главного модуля main.py последовательно вызываются configure_logging, создание разделяемого ParkingDetector (набор весов YOLOv8s загружается в память один раз на все шесть камер) и init_barrier_controllers с eager-загрузкой PlateScanner - холодный старт YOLO11x+ParseQ ~10 секунд амортизируется до первого обращения.

3.8.2. REST API и WebSocket-интерфейс

Интерфейс сервера состоит из одиннадцати HTTP-маршрутов и одного WebSocket-канала. Потокная группа - GET /, GET /api/stats, POST /api/frame, WS /ws/stream; барьерная - GET /api/barrier/log, GET/POST/DELETE /api/barrier/plates, GET /api/barrier/{id}, POST /api/barrier/{id}/ue5_ack, POST /api/barrier/{id}/manual. Ядро - WebSocket /ws/stream: при подключении запускается 10-герцовый цикл, который для каждой из шести камер читает последний кадр из SmartParking/frames/, пропускает его через ParkingDetector и, для барьерных, через BarrierController; обработанное изображение кодируется в JPEG качества 80, упаковывается в base64 и вместе с агрегированной статистикой и состоянием барьера отправляется клиенту одним JSON-сообщением. REST-маршруты барьера дублируют подмножество тех же данных для UE5, который опрашивает /api/barrier/{id} на частоте 1 Гц и подтверждает завершение анимаций через ue5_ack.

3.8.3. Типизированная конфигурация

Все настраиваемые параметры - от порога уверенности YOLO до таймеров барьера - централизованы в пакете `neosmart.config`. Корневая модель `AppSettings` (`pydantic-settings BaseSettings`) агрегирует семь подсистем: `PathsSettings`, `CameraSettings` (словарь по идентификатору камеры), `DetectorSettings`, `TrackingSettings`, `BarrierSettings`, `WrongParkingSettings`, `LoggingSettings`. Источники значений разрешаются в строгом порядке от более приоритетного к менее: `init`-аргументы в коде, переменные окружения `NEOSMART_*` с разделителем `__` для вложенных полей, файл `.env`, `config/default.yaml` с опциональным наложением `config/<profile>.yaml` (профиль выбирается переменной `NEOSMART_CONFIG_PROFILE`), значения по умолчанию в самих моделях. Валидаторы отклоняют некорректные значения на старте, а не в `runtime`.

3.8.4. Структурированное логирование

Модуль `neosmart.logging_setup` устанавливает на корневой логгер два обработчика: консольный в `stderr` в человекочитаемом формате и файловый в `logs/neosmart.jsonl` в виде одной JSON-записи на строку с полями `ts`, `level`, `logger`, `msg`, `exc` и произвольными `extra`-полями. Такой формат без дополнительного парсинга читается типовыми коллекторами. Повторный вызов `configure_logging` идемпотентен благодаря атрибуту-стражу на корневом логгере, а уровень шумящих сторонних логгеров (`urllib3`, `PIL`, `matplotlib`, `httpx`, `httpcore`) принудительно понижается до `WARNING`.

3.9. Клиентская часть

3.9.1. Архитектура фронтенд-приложения

Клиентская часть - одностраничное приложение без фреймворков, построенное вокруг класса `ParkingMonitor` (`static/js/app.js`). Конструктор связывает шесть `camera`-объектов с их DOM-элементами (`canvas` для оригинального и обработанного кадра, значки статуса, счётчики), объявляет

четыре зоны - free (camera1, camera2), paid (camera3), road (camera4), barrier (camera5, camera6) - и устанавливает WebSocket-соединение с /ws/stream. Входящие сообщения декодируются в base64-кадры, которые через объект Image загружаются в canvas: каждая камера отрисовывается только в момент, когда её вкладка активна, что снижает нагрузку на GPU браузера. Переключение между видами (overview, free, paid, road, barrier, access) - без перезагрузки страницы, а выбранный вид сохраняется в localStorage и восстанавливается при следующем открытии.

3.9.2. Агрегация статистики и дедупликация камер

Поскольку camera1 и camera2 наблюдают одну и ту же бесплатную зону с разных ракурсов, простое суммирование их счётчиков давало бы двойной учёт автомобилей. Клиент и сервер согласованно вводят правило дедупликации: в общий показатель «машин в бесплатной зоне» включается только значение camera1, тогда как camera2 используется для повышения покрытия распознавания на границах полигонов. Занятость парковки (available / occupied / total) агрегируется по объединению свободных и занятых полигонов двух камер: идентификаторы мест (camera_id, index) позволяют корректно свести пересекающиеся разметки. Для платной зоны и дорожной сцены агрегация прямая - по одной камере на зону; барьерная зона агрегирует только состояние автомата, а не счётчик машин.

3.9.3. Панель управления шлагбаумом и журнал доступа

Барьерная вкладка объединяет три панели. Панель состояния отображает текущее состояние автомата, последний распознанный номер и флаг access=granted|denied. Панель ручного управления содержит кнопки open и close, отправляющие POST на /api/barrier/entry/manual - они используются оператором в аварийных ситуациях и обходят whitelist-проверку с отметкой manual_override в журнале. Панель whitelist предоставляет CRUD-интерфейс над таблицей allowed_plates: метод addPlate валидирует ввод и отправляет

POST /api/barrier/plates, метод removePlate - DELETE /api/barrier/plates/{plate}. Журнал доступа обновляется на каждом WebSocket-сообщении: метод renderAccessLog перерисовывает последние двадцать записей с цветовой кодировкой результата и превью сохранённого кадра.

3.10. Инфраструктура развёртывания и обеспечение качества

3.10.1. Контейнеризация и оркестрация

Dockerfile собирает образ на основе python:3.11-slim в несколько шагов кэшируемых слоёв: системные библиотеки libGL и glib для opencv-python, установка torch 2.5.1+cpu с индекса PyTorch (CPU-колесо на ~400 МБ вместо ~2 ГБ CUDA-варианта), затем requirements и, отдельным слоем, исходный код пакета. Запуск выполняется под непривилегированным пользователем с UID 1000 - это совпадает с типовым UID разработчика и избавляет от ручного chown на bind-mounted томах. HEALTHCHECK периодически опрашивает /api/stats, избегая дорогой инициализации детектора.

Оркестрация описана в docker-compose.yml. Каталог frames/ монтируется в контейнер read-only (UE5 пишет туда на хосте), Models/ и калибровочные каталоги (CarParkingSpace, BarrierSystem, TrafficCounting) - тоже read-only; logs/ - на запись, чтобы JSONL-логи переживали перезапуск; база данных барьера размещена в именованном томе barrier_db, который сохраняется при docker compose down и очищается только при down -v. Параллельно существует docker-compose.clearml.yml - шестиконтейнерный стенд ClearML для обучения, независимый от производственного приложения.

3.10.2. Непрерывная интеграция

CI-конвейер реализован в GitHub Actions (.github/workflows/ci.yml) и состоит из двух задач, связанных зависимостью needs. Первая задача lint-and-test в ~15 минут выполняет на Ubuntu-runner: checkout репозитория с submodules (PlateScanner-весы обязательны), установку Python 3.11 с кэшированием pip по обоим файлам requirements, установку системных

библиотек для opencv-python, установку torch-CPU и проекта в editable-режиме, затем ruff check и ruff format --check, мурку в режиме warn-only (continue-on-error: true - строгий режим включён для neosmart.* в pyproject.toml и будет задействован после вычистки легаси-модулей), pytest с покрытием. Вторая задача docker-build выполняется после успешного прохождения первой и собирает образ через docker buildx с кэшированием слоёв в gha-backend, что доказывает работоспособность Dockerfile на каждом pull request. Конкуренция настроена на отмену устаревших запусков в пределах одной ветки.

3.10.3. Модульное и интеграционное тестирование

Тестовая сьюта pytest содержит 55 тестов, распределённых по шести файлам: test_config.py (загрузка и приоритет источников конфигурации, валидаторы ролей камер), test_sort.py (детерминизм трекера, матчинг по IoU, сохранение идентификатора при кратковременных окклюзиях, уничтожение трека после max_age), test_barrier_state_machine.py (все переходы конечного автомата, немедленный grant по whitelist, консенсус при denied-решении, окно ожидания safety), test_barrier_db.py (CRUD whitelist, журнал доступа, жизненный цикл parking_sessions), test_wrong_parking.py (геометрия bbox-полигон на восьми граничных случаях), test_api_smoke.py (FastAPI TestClient на ключевых маршрутах). Покрытие измеряется плагином coverage на пакете neosmart; локально пороговое значение не закреплено, но отчёт term-missing отображается в каждом CI-прогоне. Общее время исполнения сьюты - около 28 секунд, что позволяет запускать её вручную перед каждым коммитом.

Заключение

В работе построена полнофункциональная система интеллектуального управления и мониторинга парковочного пространства, в которой задача компьютерного зрения доведена от исследовательской постановки до продуктового сервиса, устойчивого к реальным нагрузкам демонстрационного стенда. Все восемь задач, сформулированных во введении, получили конкретные технические ответы.

Аналитический обзор современных подходов к детекции, многообъектному трекингу и распознаванию регистрационных знаков показал, что оптимальный по соотношению точности и скорости выбор для данной сцены - одностадийный детектор семейства YOLO в связке с классическим SORT-трекером. Обоснование отказа от DeepSORT, ByteTrack и StrongSORT получено на уровне анализа вычислительных затрат и рисков на стационарных камерах с видом сверху.

Виртуальный полигон в Unreal Engine 5 воспроизводит участок улицы Богородского и территорию торгового центра «Нагорный» в Нижнем Новгороде. В сцене функционируют автономные агенты-автомобили с конечным автоматом поиска места и двумя стратегиями парковки, пешеходы и динамический день-ночь-цикл. Шесть камер, размещённых в соответствии с зональной архитектурой, экспортируют кадры в рабочую директорию backend-сервера и взаимодействуют с ним по REST-протоколу, формирующему замкнутый контур управления шлагбаумом.

Синтетический датасет из 822 кадров и 1821 аннотации класса car собран из этого виртуального полигона и размечен в CVAT. Разбиение 70/20/10 по train/val/test проверено на отсутствие утечки, а Колмогоров-Смирнов-тест подтвердил сопоставимость распределений геометрии рамок между подвыборками. Сопроводительные Data Card и Model Card фиксируют границы применимости модели.

На этом датасете обучены и сравнены три конфигурации YOLOv8s. Baseline достигает $mAP50 = 0.9945$ и $mAP50-95 = 0.9738$, а более агрессивные аугментации и оптимизатор AdamW не дают статистически значимого прироста на верхнем участке кривой. Все три прогона логгировались локальным ClearML-сервером из шести контейнеров; продакшн-чекпоинт закреплён как Models/Car_Detector.pt. Модель распознавания регистрационных знаков реализована собственной сборкой PlateScanner на базе обученного автором детектора YOLO11x и open-source OCR-декодера ParseQ, с конвертацией весов в TFLite FP16 для будущего edge-развёртывания.

Серверная часть на FastAPI объединяет шесть CV-модулей в одном процессе: детекция и учёт занятости по полигонам, обнаружение неправильной парковки с адаптивными порогами для fisheye-камер, SORT-трекинг с таймером подозрительной стоянки, отдельный подсчёт встречного и попутного трафика, двухкамерное управление входным шлагбаумом с конечным автоматом из девяти состояний и асимметричной консенсусной логикой OCR. Клиентское приложение без фреймворков отрисовывает потоки со всех шести камер через WebSocket на частоте 10 Гц, агрегирует статистику по четырём зонам с дедупликацией парных ракурсов и предоставляет панель администрирования whitelist и журнала доступа.

Качество и воспроизводимость инфраструктуры обеспечены тремя связанными механизмами: Dockerfile и docker-compose.yml с bind-mount-томами для кадров, моделей и логов; GitHub Actions с двумя задачами lint-and-test и docker-build, выполняющими ruff, mypy в warn-only-режиме, pytest с покрытием и проверку сборки образа на каждом pull request; тестовая сьюта из 55 тестов в шести файлах, покрывающая загрузку конфигурации, детерминизм SORT-трекера, граф переходов барьерного автомата, CRUD whitelist, геометрию неправильной парковки и smoke-контракты REST-маршрутов FastAPI.

Практическая значимость работы подтверждается тем, что система развёртывается одной командой, обрабатывает шесть параллельных на CPU-борке torch и не требует дополнительных ручных операций после первого запуска. Подход к валидации на цифровом двойнике позволяет измерять метрики модулей без разметки реального видео - это существенно сокращает стоимость поддержки системы при расширении на новые локации.

Вместе с тем работа имеет ограничения, которые необходимо озвучить. Все приведённые метрики получены на синтетических данных из UE5-сцены; перенос модели на реальные камеры торгового центра потребует дообучения на смешанной выборке или domain adaptation по схеме CycleGAN с последующей валидацией на полностью независимом реальном тесте. Порог confidence детектора 0.65 и геометрические пороги неправильной парковки также калибровались под конкретную установку камер и будут нуждаться в повторной настройке. Кроме того, текущая реализация не содержит ночного режима с инфракрасными камерами, хотя архитектура конфигурации и модуля детекции не препятствует такому расширению.

Перспективными направлениями развития представляются: интеграция с муниципальными сервисами мобильной оплаты по API Российского оператора автоматизированной системы безналичной оплаты; перенос inference-части на edge-устройства класса NVIDIA Jetson или Raspberry Pi 5 - квантизированные веса PlateScanner уже подготовлены; расширение барьерной подсистемы на выезд с двунаправленной биллинг-логикой; пилотное развёртывание на реальной паре камер с параллельной работой обеих сред для валидации переноса модели. Базис, сформированный настоящей работой, делает каждое из этих направлений инкрементальным улучшением, а не переработкой с нуля.

Библиографический список

1. Redmon J., Divvala S., Girshick R., Farhadi A. You Only Look Once: Unified, Real-Time Object Detection // *Proc. of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016. P. 779–788. DOI: 10.1109/CVPR.2016.91. URL: <https://arxiv.org/abs/1506.02640>.
2. Redmon J., Farhadi A. YOLOv3: An Incremental Improvement // *arXiv preprint arXiv:1804.02767*. 2018. URL: <https://arxiv.org/abs/1804.02767>.
3. Bochkovskiy A., Wang C.-Y., Liao H.-Y. M. YOLOv4: Optimal Speed and Accuracy of Object Detection // *arXiv preprint arXiv:2004.10934*. 2020. URL: <https://arxiv.org/abs/2004.10934>.
4. Jocher G., Chaurasia A., Qiu J. Ultralytics YOLOv8 // *Ultralytics Documentation*. 2023. URL: <https://docs.ultralytics.com/models/yolov8/>.
5. Ren S., He K., Girshick R., Sun J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks // *Advances in Neural Information Processing Systems (NeurIPS)*. 2015. P. 91–99. URL: <https://arxiv.org/abs/1506.01497>.
6. Liu W., Anguelov D., Erhan D., Szegedy C., Reed S., Fu C.-Y., Berg A. C. SSD: Single Shot MultiBox Detector // *European Conference on Computer Vision (ECCV)*. 2016. P. 21–37. DOI: 10.1007/978-3-319-46448-0_2. URL: <https://arxiv.org/abs/1512.02325>.
7. Zhao Y., Lv W., Xu S., Wei J., Wang G., Dang Q., Liu Y., Chen J. DETRs Beat YOLOs on Real-time Object Detection // *Proc. of IEEE/CVF CVPR*. 2024. URL: <https://arxiv.org/abs/2304.08069>.
8. Bewley A., Ge Z., Ott L., Ramos F., Upcroft B. Simple Online and Realtime Tracking // *Proc. of IEEE International Conference on Image Processing (ICIP)*. 2016. P. 3464–3468. DOI: 10.1109/ICIP.2016.7533003. URL: <https://arxiv.org/abs/1602.00763>.
9. Wojke N., Bewley A., Paulus D. Simple Online and Realtime Tracking with a Deep Association Metric // *Proc. of IEEE International Conference on Image Processing (ICIP)*. 2017. P. 3645–3649. DOI: 10.1109/ICIP.2017.8296962. URL: <https://arxiv.org/abs/1703.07402>.
10. Zhang Y., Sun P., Jiang Y., Yu D., Weng F., Yuan Z., Luo P., Liu W., Wang X. ByteTrack: Multi-Object Tracking by Associating Every Detection Box // *European Conference on Computer Vision (ECCV)*. 2022. URL: <https://arxiv.org/abs/2110.06864>.

11. Du Y., Zhao Z., Song Y., Zhao Y., Su F., Gong T., Meng H. StrongSORT: Make DeepSORT Great Again // *IEEE Transactions on Multimedia*. 2023. URL: <https://arxiv.org/abs/2202.13514>.
12. Cao J., Pang J., Weng X., Khirodkar R., Kitani K. Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking // *Proc. of IEEE/CVF CVPR*. 2023. URL: <https://arxiv.org/abs/2203.14360>.
13. Kuhn H. W. The Hungarian Method for the Assignment Problem // *Naval Research Logistics Quarterly*. 1955. Vol. 2, № 1–2. P. 83–97. DOI: 10.1002/nav.3800020109. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109>.
14. Kalman R. E. A New Approach to Linear Filtering and Prediction Problems // *Transactions of the ASME — Journal of Basic Engineering*. 1960. Vol. 82, Series D. P. 35–45. URL: <https://asmedigitalcollection.asme.org/fluidsengineering/article/82/1/35/397706>.
15. Bautista D., Atienza R. Scene Text Recognition with Permuted Autoregressive Sequence Models // *European Conference on Computer Vision (ECCV)*. 2022. DOI: 10.1007/978-3-031-19815-1_11. URL: <https://arxiv.org/abs/2207.06966> (дата обращения: 24.04.2026).
16. Zherzdev S., Gruzdev A. LPRNet: License Plate Recognition via Deep Neural Networks // *arXiv preprint arXiv:1806.10447*. 2018. URL: <https://arxiv.org/abs/1806.10447>.
17. Du S., Ibrahim M., Shehata M., Badawy W. Automatic License Plate Recognition (ALPR): A State-of-the-Art Review // *IEEE Transactions on Circuits and Systems for Video Technology*. 2013. Vol. 23, № 2. P. 311–325. DOI: 10.1109/TCSVT.2012.2203741. URL: <https://ieeexplore.ieee.org/document/6213519/>.
18. Amato G., Carrara F., Falchi F., Gennaro C., Meghini C., Vairo C. Deep learning for decentralized parking lot occupancy detection // *Expert Systems with Applications*. 2017. Vol. 72. P. 327–334. DOI: 10.1016/j.eswa.2016.10.055. URL: <https://www.sciencedirect.com/science/article/abs/pii/S095741741630598X>.
19. Amato G., Carrara F., Falchi F., Gennaro C., Vairo C. Car parking occupancy detection using smart camera networks and Deep Learning // *Proc. of IEEE Symposium on Computers and Communication (ISCC)*. 2016. P. 1212–1217. DOI: 10.1109/ISCC.2016.7543901. URL: <https://ieeexplore.ieee.org/document/7543901/>.
20. De Almeida P. R. L., Oliveira L. S., Britto Jr. A. S., Silva Jr. E. J., Koerich A. L. PKLot — A robust dataset for parking lot classification // *Expert Systems with Applications*. 2015. Vol. 42, № 11. P. 4937–4949. DOI: 10.1016/j.eswa.2015.02.009. URL: <https://www.sciencedirect.com/science/article/abs/pii/S0957417415001086>.

21. Acharya D., Yan W., Khoshelham K. Real-time image-based parking occupancy detection using deep learning // *Proc. of the 5th Annual Conference of Research@Locate, CEUR Workshop Proceedings*. 2018. Vol. 2087. P. 33–40. URL: <https://ceur-ws.org/Vol-2087/paper5.pdf>.
22. Tobin J., Fong R., Ray A., Schneider J., Zaremba W., Abbeel P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World // *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017. P. 23–30. DOI: 10.1109/IROS.2017.8202133. URL: <https://arxiv.org/abs/1703.06907>.
23. Tremblay J., Prakash A., Acuna D., Brophy M., Jampani V., Anil C., To T., Cameracci E., Bochoon S., Birchfield S. Training Deep Networks with Synthetic Data: Bridging the Reality Gap by Domain Randomization // *Proc. of IEEE/CVF CVPR Workshops*. 2018. URL: <https://arxiv.org/abs/1804.06516>.
24. Turkcan M. K., Li Y., Zang C., Ghaderi J., Zussman G., Kostic Z. Boundless: Generating Photorealistic Synthetic Data for Object Detection in Urban Streetscapes // *arXiv preprint arXiv:2409.03022*. 2024. URL: <https://arxiv.org/abs/2409.03022>.
25. Ramírez S. FastAPI: Modern, fast (high-performance), web framework for building APIs with Python // *Official documentation*. 2024. URL: <https://fastapi.tiangolo.com/>.
26. Merkel D. Docker: Lightweight Linux Containers for Consistent Development and Deployment // *Linux Journal*. 2014. Vol. 2014, № 239. URL: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
27. ClearML Authors (Allegro AI). ClearML — Auto-Magical CI/CD to Streamline Your AI Workload // *Open-source repository, GitHub*. 2024. URL: <https://github.com/clearml/clearml>.
28. Szeliski R. Computer Vision: Algorithms and Applications. 2nd ed. — Cham: Springer, 2022. — 941 p. — (Texts in Computer Science). — DOI: 10.1007/978-3-030-34372-9. URL: <https://szeliski.org/Book/>.
29. Gonzalez R. C., Woods R. E. Digital Image Processing. 4th ed. — New York: Pearson, 2018. — 1192 p. — ISBN 978-0-13-335672-4. URL: <https://www.pearson.com/en-us/subject-catalog/p/Gonzalez-Digital-Image-Processing-4th-Edition/P200000003224>.
30. Lin T.-Y., Maire M., Belongie S., Bourdev L., Girshick R., Hays J., Perona P., Ramanan D., Zitnick C. L., Dollár P. Microsoft COCO: Common Objects in Context // *European*

Conference on Computer Vision (ECCV). 2014. P. 740–755. DOI: 10.1007/978-3-319-10602-1_48. URL: <https://arxiv.org/abs/1405.0312>.

31. ГОСТ Р 52289-2019. Технические средства организации дорожного движения. Правила применения дорожных знаков, разметки, светофоров, дорожных ограждений и направляющих устройств: национальный стандарт Российской Федерации: утв. Приказом Росстандарта от 20.12.2019 г. № 1425-ст. — М.: Стандартиформ, 2020. URL: <https://base.garant.ru/73728515/>.

Приложение А

Дополнительные скриншоты виртуальной сцены и веб-интерфейса

В приложении собраны вспомогательные визуальные материалы, не вошедшие в основной текст работы. Они показывают дополнительные ракурсы сцены Unreal Engine 5, схему агрегации данных в дашборде и ключевые пайплайны, на которые опирается реализация модулей главы 3.

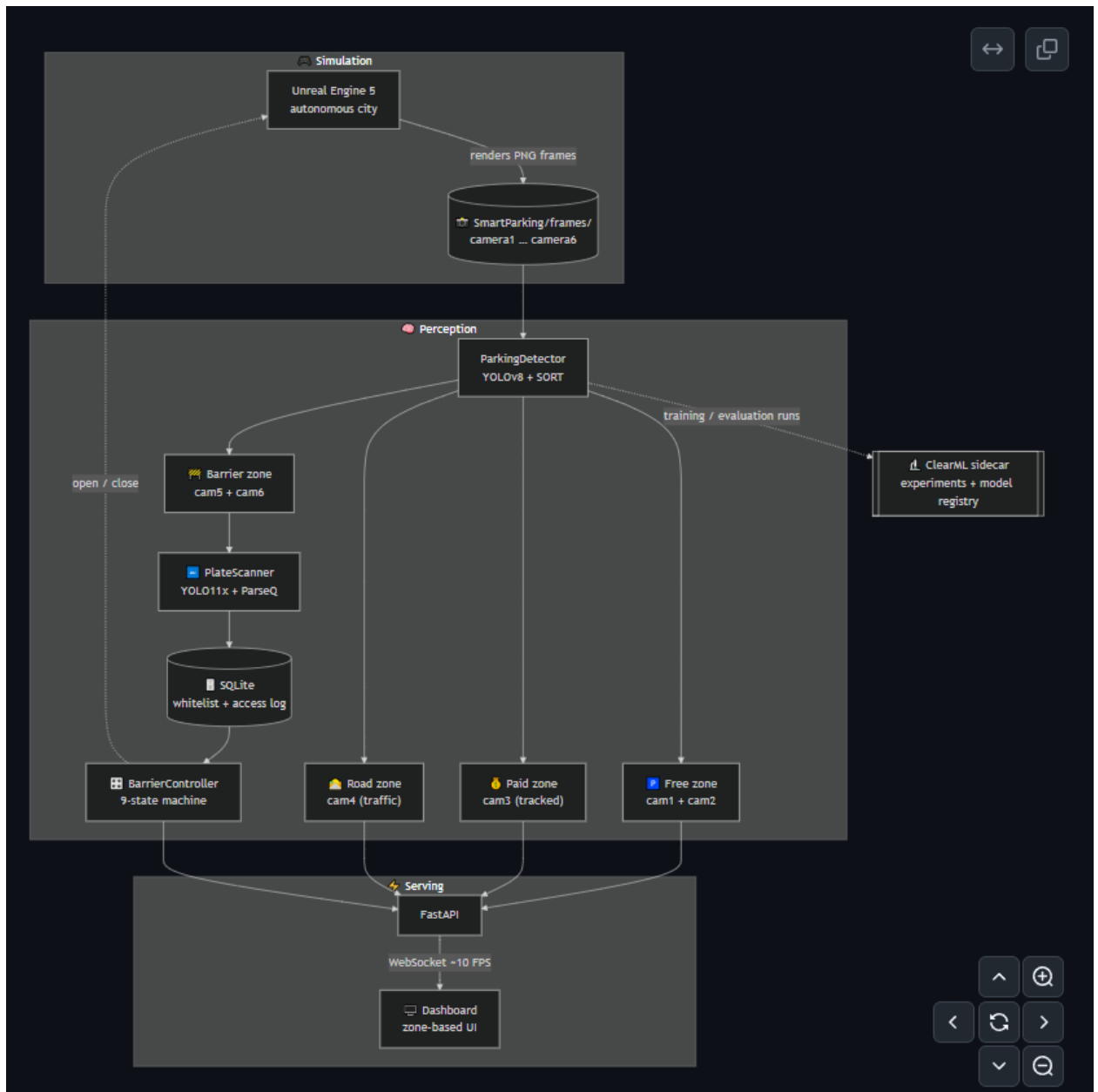


Рис. А.1. Общая архитектура приложения.

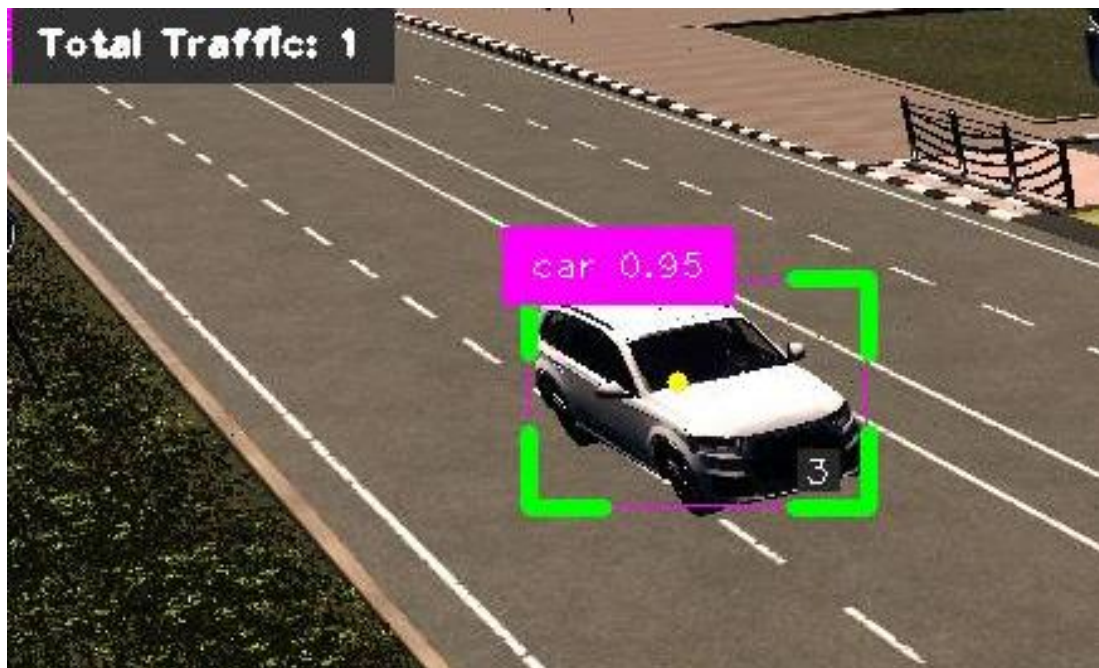


Рис. А.2. Пайплайн детекции и трекинга автомобилей.

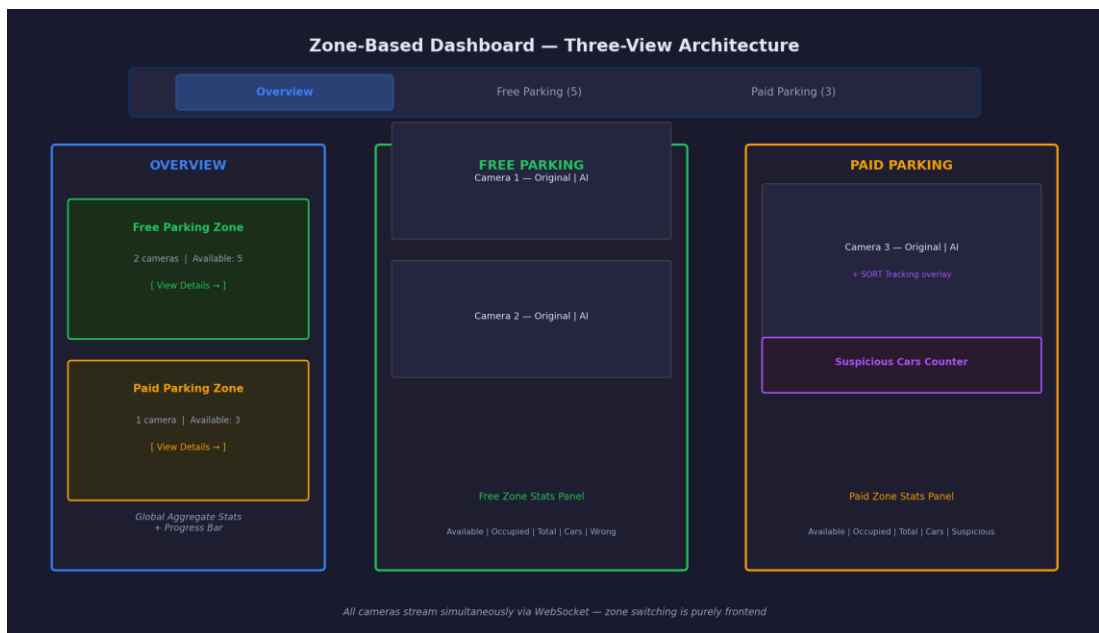


Рис. А.3. Зонированные представления дашборда: обзор, свободная и платная зоны.

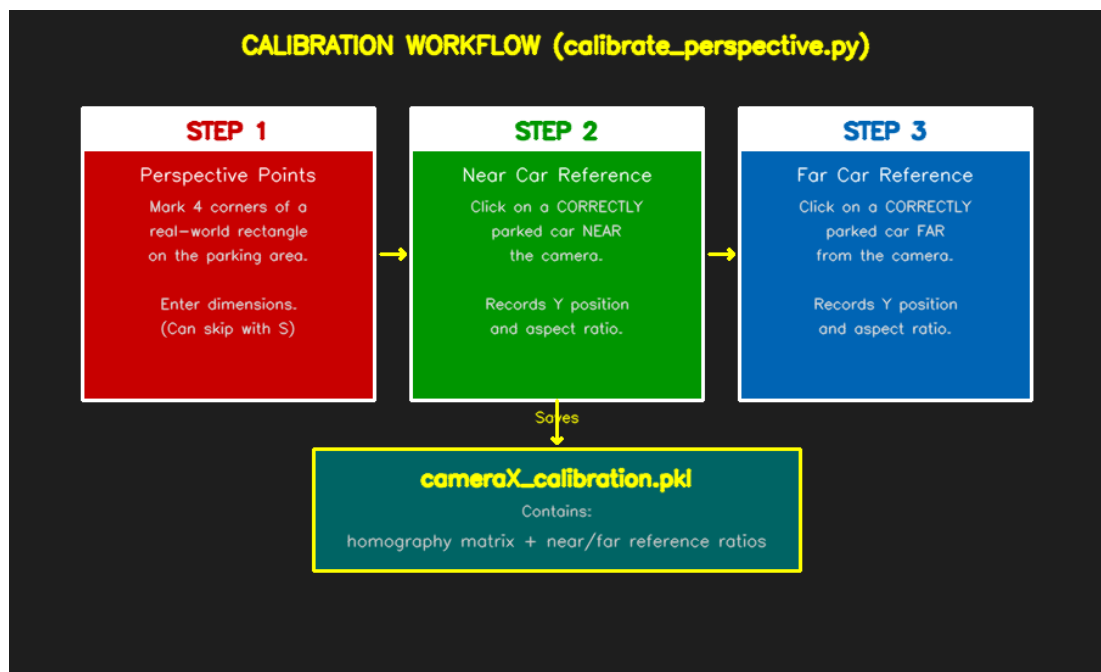


Рис. А.4. Процесс калибровки полигонов парковочных мест.

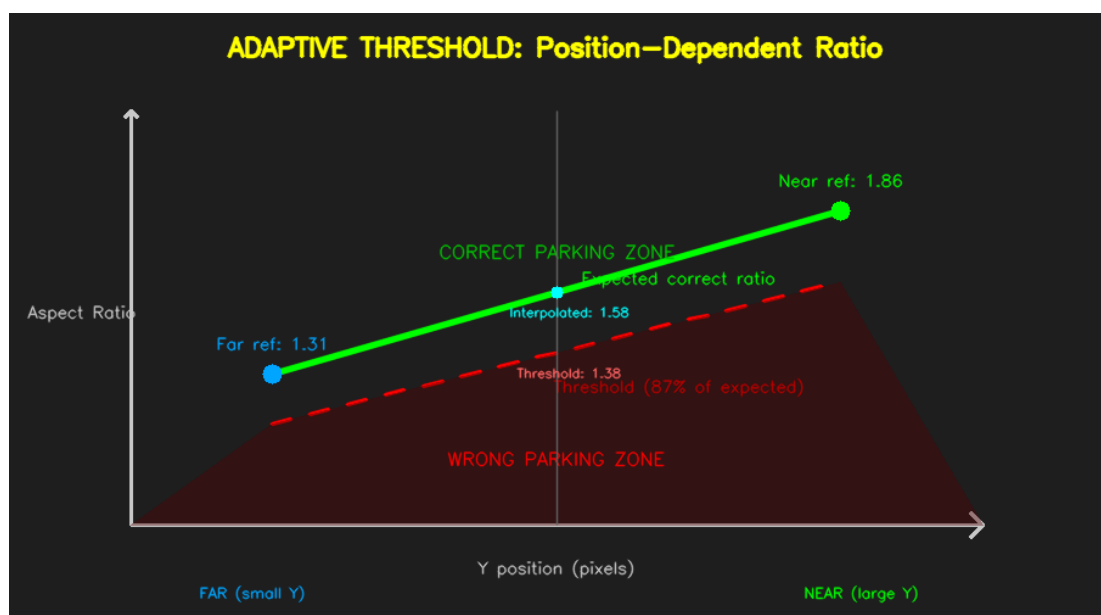


Рис. А.5. Адаптивные пороги детекции неправильной парковки для краевых полигонов.

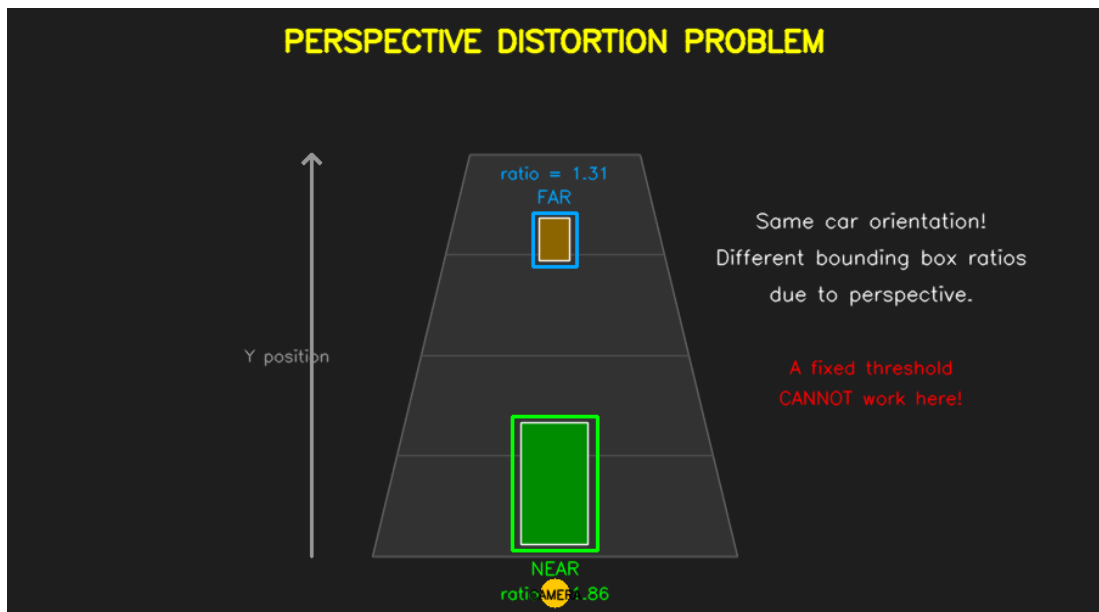


Рис. А.6. Перспективные искажения fisheye-объектива и их влияние на проекцию bbox.

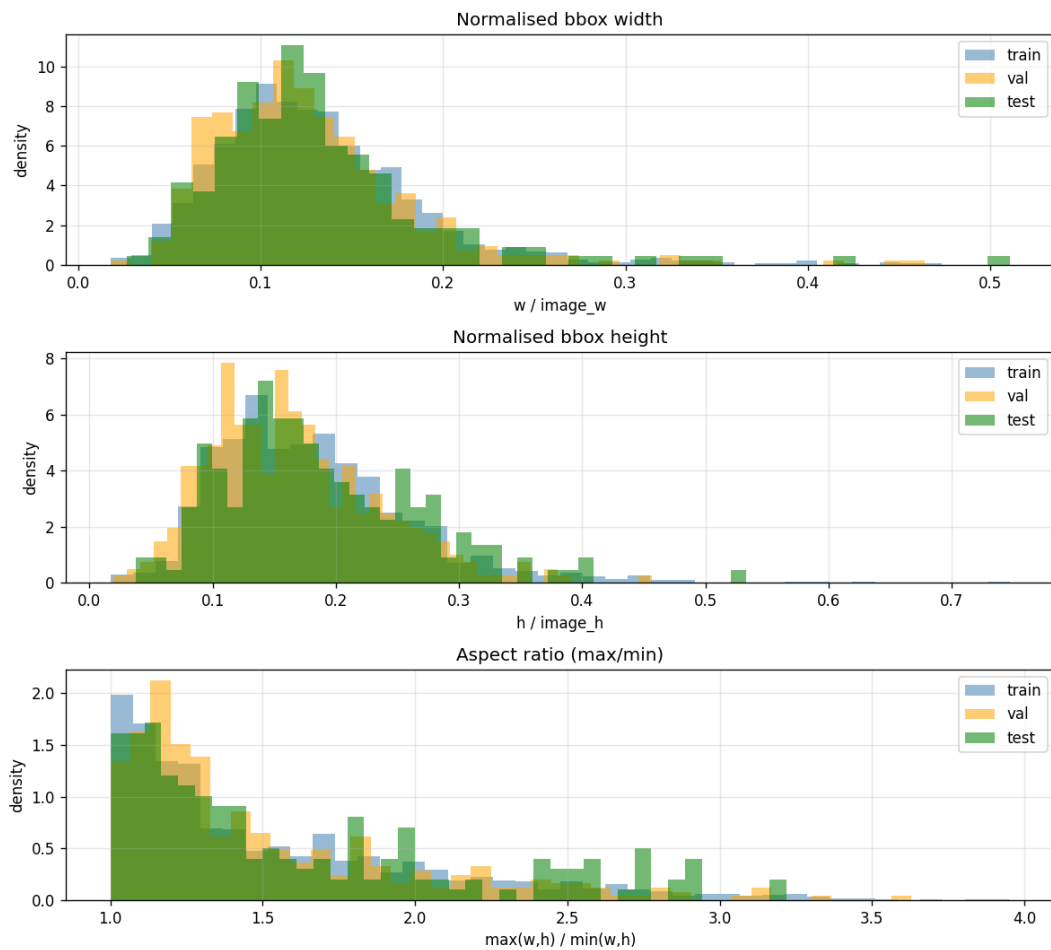


Рис. А.7. Геометрия bounding-box: ширина, высота, соотношение сторон, площадь.

val — GT sample grid



Рис. А.8. Фрагмент валидационной выборки с Ground Truth.

Приложение Б

Расширенные таблицы результатов экспериментов

В таблице Б.1 приведены сводные метрики трёх запусков обучения, описанных в разделе 2.3. Для каждого эксперимента указаны значения mAP50, mAP50-95, точности, полноты, F1-метрики при пороге уверенности 0,5, а также номер эпохи, на которой достигнута лучшая модель. Нумерация экспериментов соответствует идентификаторам ClearML.

Таблица Б.1. Сводные метрики экспериментов на валидационной выборке

№	Эксперимент	mAP50	mAP50-95	Precision	Recall	F1@0.5	Лучшая эпоха
01	baseline_yolov8s_100ep	0.9945	0.9738	0.9942	0.9920	0.9931	98/100
02	aug_schedule	0.9946	0.9728	0.9943	0.9866	0.9904	109/129
03	full_stack	0.9946	0.9377	0.9940	0.9866	0.9903	90/100

Все три запуска выполнены на одном разделении датасета (576 / 164 / 82 кадра; train / val / test), на GPU NVIDIA RTX, с фиксированным random seed, при разрешении 640×640 пикселей и батче 16. Разброс mAP50-95 между baseline и full_stack составляет около 3,6 процентных пунктов; более агрессивные аугментации и замена оптимизатора на AdamW не дают выигрыша на синтетическом датасете и ухудшают обобщение на редких ракурсах.

Таблица Б.2. Конфигурация гиперпараметров экспериментов

Параметр	baseline	aug_schedule	full_stack
Оптимизатор	SGD (auto)	SGD + cosine	AdamW
Начальная скорость lr0	0.01	0.01 -> cosine	0.001
Weight decay	0.0005	0.0005	0.0005
Аугментации	default	+ mixup 0.1, copy_paste 0.1	+ extended geometric

Полные логи экспериментов, кривые валидационных метрик по эпохам и снимки веб-интерфейса ClearML сохранены в каталоге Documentation/reports/ репозитория проекта.

Приложение В

Ключевые фрагменты программного кода

Репозиторий с проектом располагается по следующей ссылке:

<https://github.com/PavelLekomtsev/NeoSmart>

Ниже приведены наиболее показательные фрагменты кода, описанные в главе 3. Полные листинги расположены в репозитории проекта; здесь оставлены функции, демонстрирующие алгоритмическое ядро каждого модуля.

Листинг В.1. IoU-ассоциация и линейное присваивание в трекаре SORT

```
def iou_batch(bb_test, bb_gt):
    bb_gt = np.expand_dims(bb_gt, 0)
    bb_test = np.expand_dims(bb_test, 1)
    xx1 = np.maximum(bb_test[..., 0], bb_gt[..., 0])
    yy1 = np.maximum(bb_test[..., 1], bb_gt[..., 1])
    xx2 = np.minimum(bb_test[..., 2], bb_gt[..., 2])
    yy2 = np.minimum(bb_test[..., 3], bb_gt[..., 3])
    w = np.maximum(0., xx2 - xx1)
    h = np.maximum(0., yy2 - yy1)
    wh = w * h
    area_a = (bb_test[..., 2] - bb_test[..., 0]) * (bb_test[..., 3] - bb_test[...
1])
    area_b = (bb_gt[..., 2] - bb_gt[..., 0]) * (bb_gt[..., 3] - bb_gt[..., 1])
    return wh / (area_a + area_b - wh)

def associate(detections, trackers, iou_threshold=0.3):
    if len(trackers) == 0:
        return np.empty((0, 2), dtype=int), np.arange(len(detections)), np.empty((0,
5), int)
    iou_matrix = iou_batch(detections, trackers)
    matched_idx = linear_assignment(-iou_matrix)
    matches = [m for m in matched_idx if iou_matrix[m[0], m[1]] >= iou_threshold]
    return np.array(matches), unmatched_dets, unmatched_trks
```

Матрица IoU между предсказаниями Калмана и текущими детекциями строится векторизованно; задача назначения сводится к максимизации суммы IoU и решается венгерским алгоритмом через обёртку `scipy.optimize.linear_sum_assignment`. Пары с IoU ниже порога отбрасываются.

Листинг В.2. Переход состояний в конечном автомате BarrierController

```

def _transition_to_reading(self, plate_text: str | None) -> None:
    if self.state != BarrierState.CAR_APPROACHING:
        return
    self._plate_readings.clear()
    if plate_text:
        self._plate_readings.append(plate_text)
    self.state = BarrierState.READING_PLATE
    self._reading_started_at = time.monotonic()

def _check_plate_consensus(self) -> str | None:
    counter = Counter(self._plate_readings)
    if not counter:
        return None
    best_plate, best_count = counter.most_common(1)[0]
    if best_count >= PLATE_MIN_AGREEING_READS:
        return best_plate
    return None

```

Алгоритм собирает OCR-гипотезы каждый кадр, пока автомобиль остаётся в зоне считывания, и разрешает переход к проверке whitelist только после накопления PLATE_MIN_AGREEING_READS=3 совпадающих прочтений. Подход устойчив к одиночным ошибкам OCR без необходимости повышать порог уверенности детектора номерной рамки.

Листинг В.3. Расчёт выхода bbox за границы полигона парковочного места

```

def outside_percentage(bbox_xyxy, polygon_xy):
    x1, y1, x2, y2 = bbox_xyxy
    bbox_area = (x2 - x1) * (y2 - y1)
    if bbox_area <= 0:
        return 100.0
    poly = ShapelyPolygon(polygon_xy)
    box = ShapelyBox(x1, y1, x2, y2)
    inter = box.intersection(poly).area
    outside = max(0.0, bbox_area - inter)
    return 100.0 * outside / bbox_area

```

Функция возвращает долю площади bbox, находящуюся вне полигона. Значение сравнивается с адаптивным порогом, выбираемым по индексу места: центральные полигоны используют OUTSIDE_THRESHOLD_DEFAULT=25 %, краевые - OUTSIDE_THRESHOLD_EDGE=45 %, а исключения по камерам задаются в карте OUTSIDE_THRESHOLD_OVERRIDES.